

ADVENTUREWORKS SALES ANALYTICS

Azure Databricks • ADLS Gen2 • Unity Catalog • AI/BI Dashboard

FINAL PROJECT REPORT & STEP-BY-STEP IMPLEMENTATION GUIDE

Project Item	Details
Platform	Microsoft Azure Databricks
Storage	Azure Data Lake Storage Gen2
Catalog	evandb_7405616343397725
Architecture	Bronze → Silver → Gold → Dashboard
Gold Model	4 dimensions + 2 fact tables
Dashboard	AdventureWorks Sales Dashboard
Automation	Databricks Job created; automation intentionally skipped
Prepared for	Project submission / GitHub documentation / interview reference

Prepared by Evan Punnen Jacob

Table of Contents

- 1. Project Overview
- 2. Architecture and Data Flow
- 3. Azure Storage Setup
- 4. Databricks and Unity Catalog Setup
- 5. Bronze Layer
- 6. Silver Layer
- 7. Gold Layer and Star Schema
- 8. Data Validation
- 9. AI/BI Dashboard
- 10. Dashboard Results
- 11. Job Automation — Intentionally Skipped
- 12. Final Project Structure
- 13. Reproduction Checklist
- 14. Security and Operational Notes
- 15. Final Completion Status
- Appendix A — Screenshot Evidence
- Appendix B — Notebook Code

1. Project Overview

This project implements an end-to-end AdventureWorks sales analytics platform using Azure Databricks and Azure Data Lake Storage Gen2. The solution follows the Medallion Architecture: raw source data is kept in Bronze, cleaned and transformed in Silver, and modeled into business-ready Gold tables. Unity Catalog is used to organize the data assets, and a Databricks AI/BI dashboard presents the final analytics.

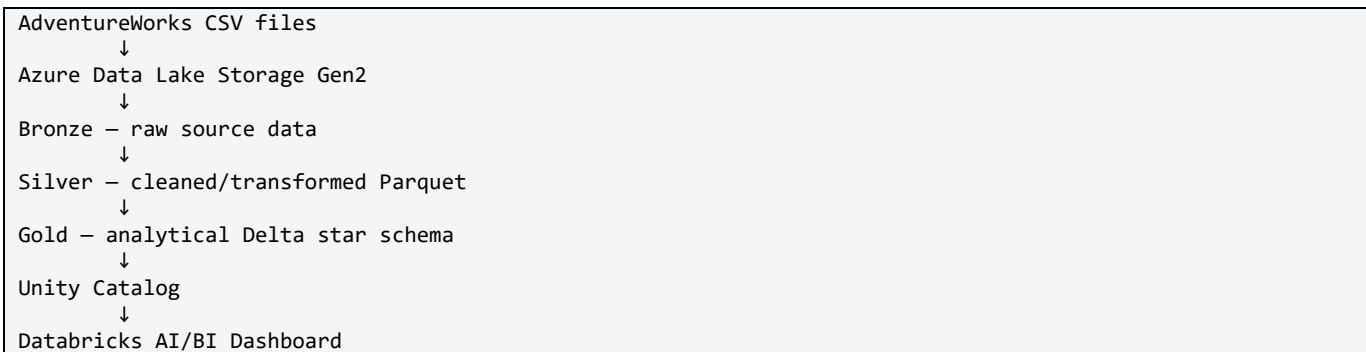
The implementation is designed for a practical data-engineering workflow: preserve the raw layer, perform controlled transformations, build a star schema, validate data quality, and expose business metrics.

Project Goals

- Store AdventureWorks source data in ADLS Gen2.
- Implement Bronze, Silver, and Gold layers.
- Transform raw data with PySpark.
- Register and organize datasets with Unity Catalog.
- Build a Gold star schema for analytics.
- Validate row counts, uniqueness, nulls, and referential integrity.
- Create an AI/BI dashboard for sales and return analysis.

2. Architecture and Data Flow

The completed analytical flow is:



Architecture Components

Layer / Component	Purpose	Status
Bronze	Preserve source CSV data	Completed
Silver	Clean and transform source data	Completed
Gold	Business-ready star schema	Completed
Unity Catalog	Catalog and govern tables	Completed
AI/BI Dashboard	Business reporting and visualization	Completed
Databricks Job	Automate Bronze → Silver → Gold	Skipped

3. Azure Storage Setup

The Azure storage account used in the project is evanpunnen. Three containers were used to separate the Medallion layers.

Storage account: evanpunnen

Containers:

bronze/

silver/

gold/

Home > evanpunnen | Containers

bronze

Container

Search

+ Add Directory Upload Refresh Delete Copy Paste Rename Acquire lease Break lease Edit columns

Authentication method: Access key (Switch to Microsoft Entra user account)

Search blobs by prefix (case-sensitive)

Only show active objects

Showing all 10 items

Name	Last modified	Access tier	Blob type	Size	Lease state
AdventureWorks_Calendar.csv	9/20/2026, 8:37:07 PM	Hot (Inferred)	Block blob	9.72 KiB	Available
AdventureWorks_Customers.csv	9/20/2026, 8:37:09 PM	Hot (Inferred)	Block blob	1.87 MiB	Available
AdventureWorks_Product_Categories.csv	9/20/2026, 8:37:06 PM	Hot (Inferred)	Block blob	83 B	Available
AdventureWorks_Product_Subcategories.csv	9/20/2026, 8:37:06 PM	Hot (Inferred)	Block blob	637 B	Available
AdventureWorks_Products.csv	9/20/2026, 8:37:07 PM	Hot (Inferred)	Block blob	56.76 KiB	Available
AdventureWorks>Returns.csv	9/20/2026, 8:37:06 PM	Hot (Inferred)	Block blob	33.78 KiB	Available
AdventureWorks_Sales_2015.csv	9/20/2026, 8:37:06 PM	Hot (Inferred)	Block blob	115.81 KiB	Available
AdventureWorks_Sales_2016.csv	9/20/2026, 8:37:07 PM	Hot (Inferred)	Block blob	1.03 MiB	Available
AdventureWorks_Sales_2017.csv	9/20/2026, 8:37:07 PM	Hot (Inferred)	Block blob	1.26 MiB	Available
AdventureWorks_Territories.csv	9/20/2026, 8:37:06 PM	Hot (Inferred)	Block blob	400 B	Available

Add or remove favorites by pressing Ctrl+Shift+F

Figure 1. Azure Storage containers showing the Bronze and Silver layer areas. (Source document page 1)

Home > evanpunnen | Containers

silver

Container

Search

+ Add Directory Upload Refresh Delete Copy Paste Rename Acquire lease Break lease Edit columns

Authentication method: Access key (Switch to Microsoft Entra user account)

Search blobs by prefix (case-sensitive)

Only show active objects

Showing all 8 items

Name	Last modified	Access tier	Blob type	Size	Lease state
calendar	9/20/2026, 9:54:46 PM				
customer	9/20/2026, 9:49:03 PM				
product_categories	9/20/2026, 10:26:37 PM				
product_subcategories	9/20/2026, 10:09:50 PM				
products	9/20/2026, 10:01:37 PM				
returns	9/20/2026, 10:19:58 PM				
sales	9/20/2026, 10:16:18 PM				
territories	9/20/2026, 10:23:17 PM				

Add or remove favorites by pressing Ctrl+Shift+F

Figure 2. Azure Storage / Gold container and related Azure resource view. (Source document page 1)

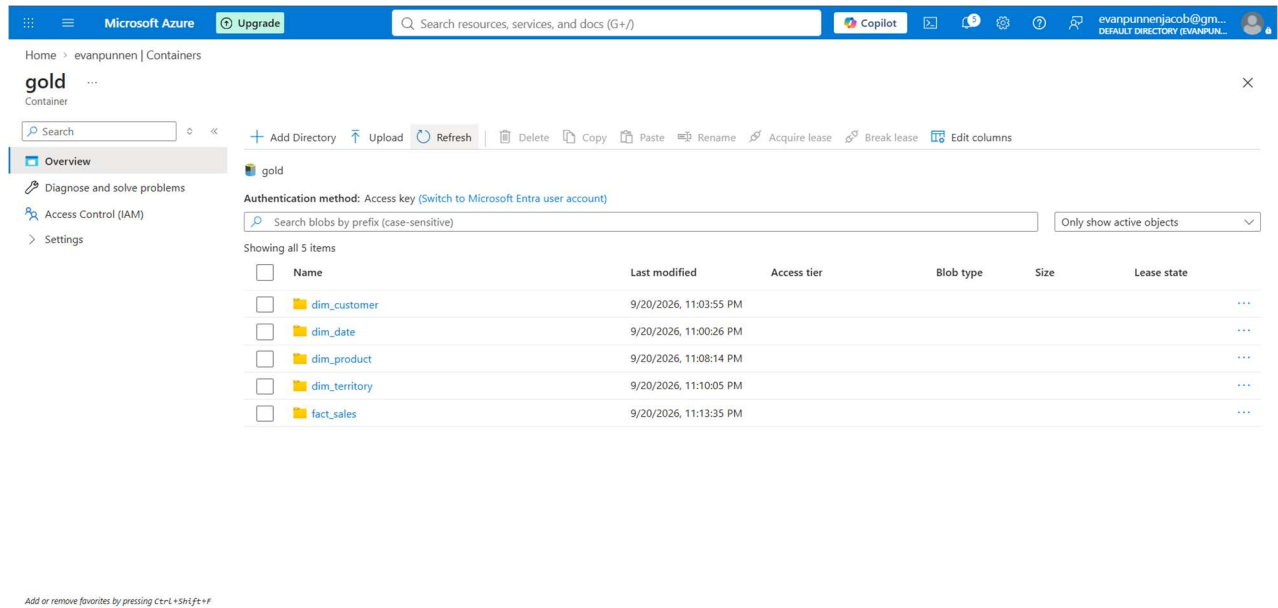


Figure 3. Gold container contents showing Gold dimension and fact directories. (Source document page 2)

Gold Storage Output

- dim_customer
- dim_date
- dim_product
- dim_territory
- fact_sales
- fact_returns

4. Databricks and Unity Catalog Setup

Unity Catalog was configured to provide a governed catalog structure for the Bronze, Silver, and Gold datasets.

4.1 Storage Credential

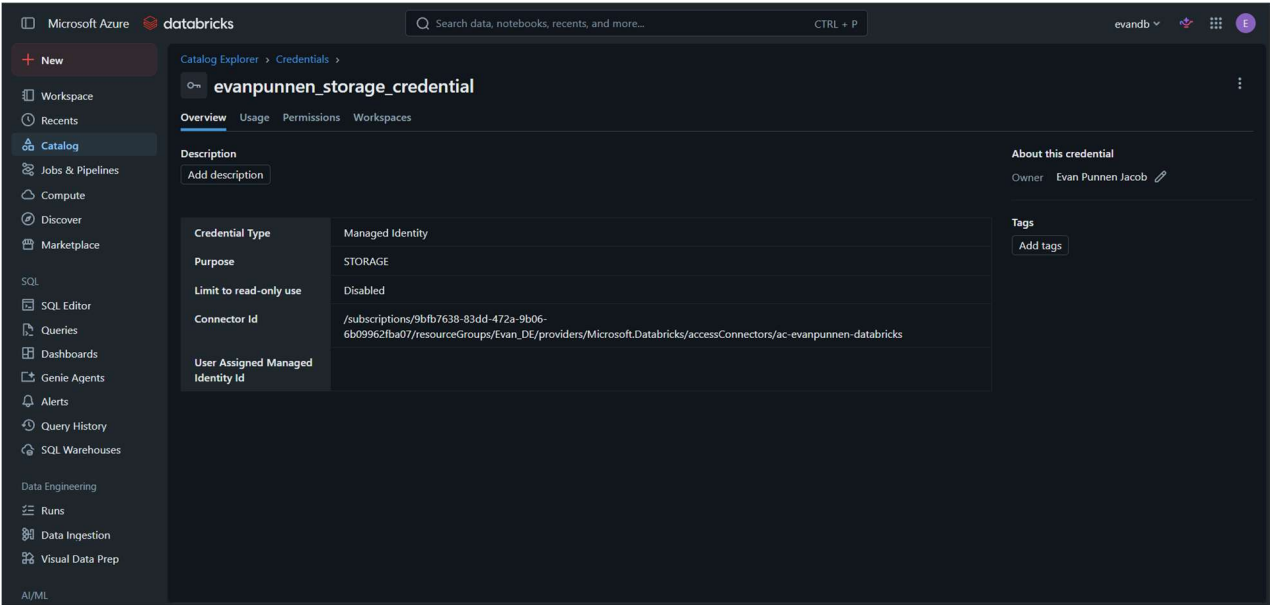


Figure 6. Databricks Unity Catalog storage credential configuration. (Source document page 3)

The project uses the Azure Access Connector / managed identity approach for storage access. A storage credential named `evanpunnen_storage_credential` was configured.

4.2 External Locations

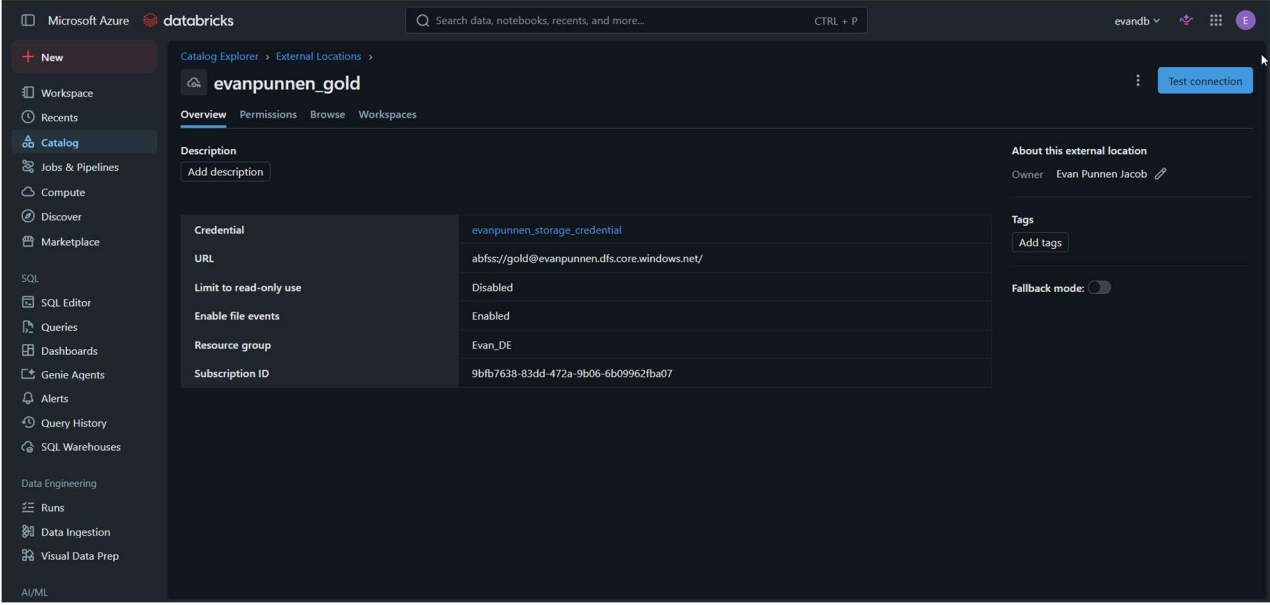


Figure 7. Unity Catalog external location for the Gold storage path. (Source document page 3)

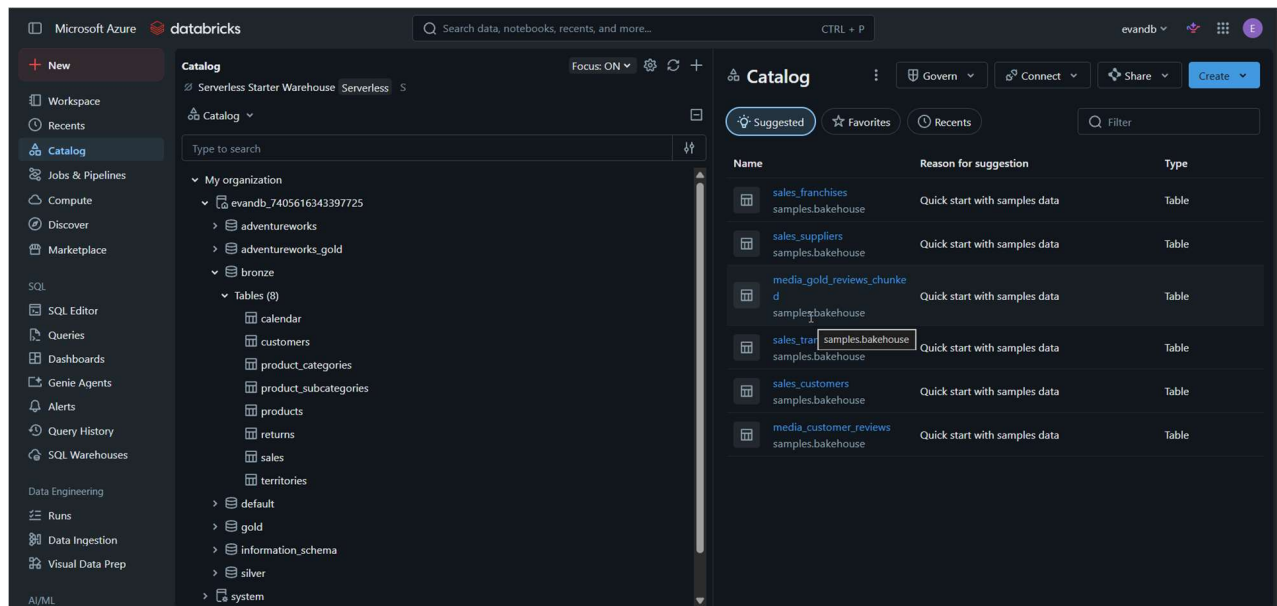


Figure 8. Unity Catalog catalog view and registered data assets. (Source document page 3)

4.3 Catalog and Schemas

Catalog:
evandb_7405616343397725

Schemas:
bronze
silver
gold

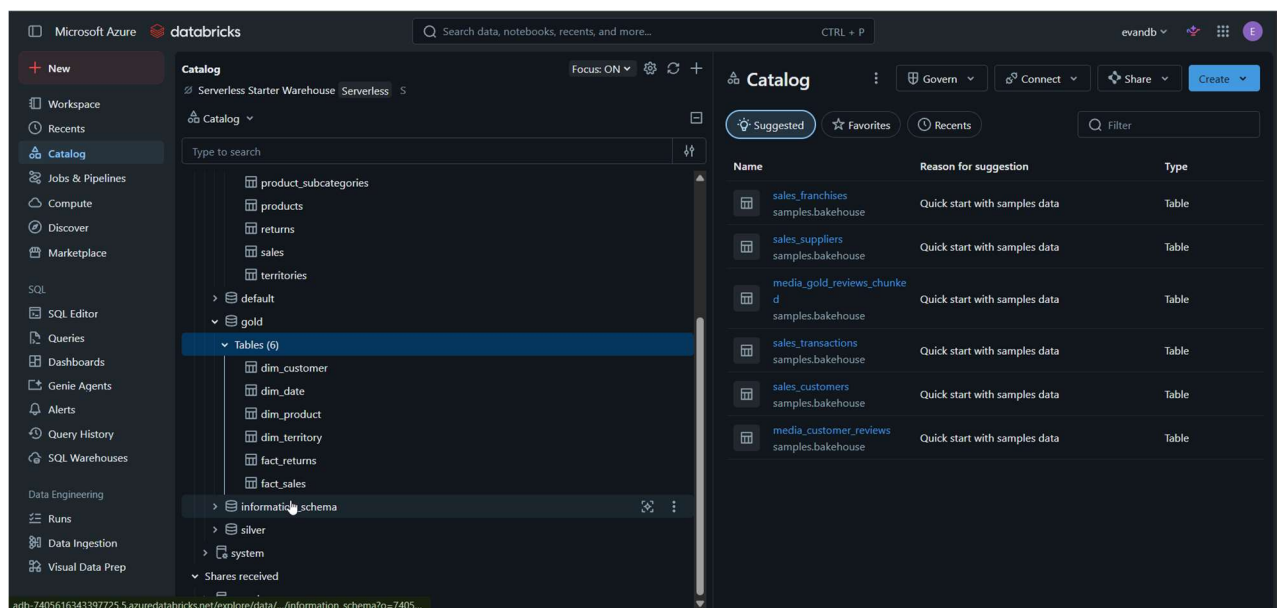


Figure 9. Unity Catalog tree showing the project catalog and schemas. (Source document page 4)

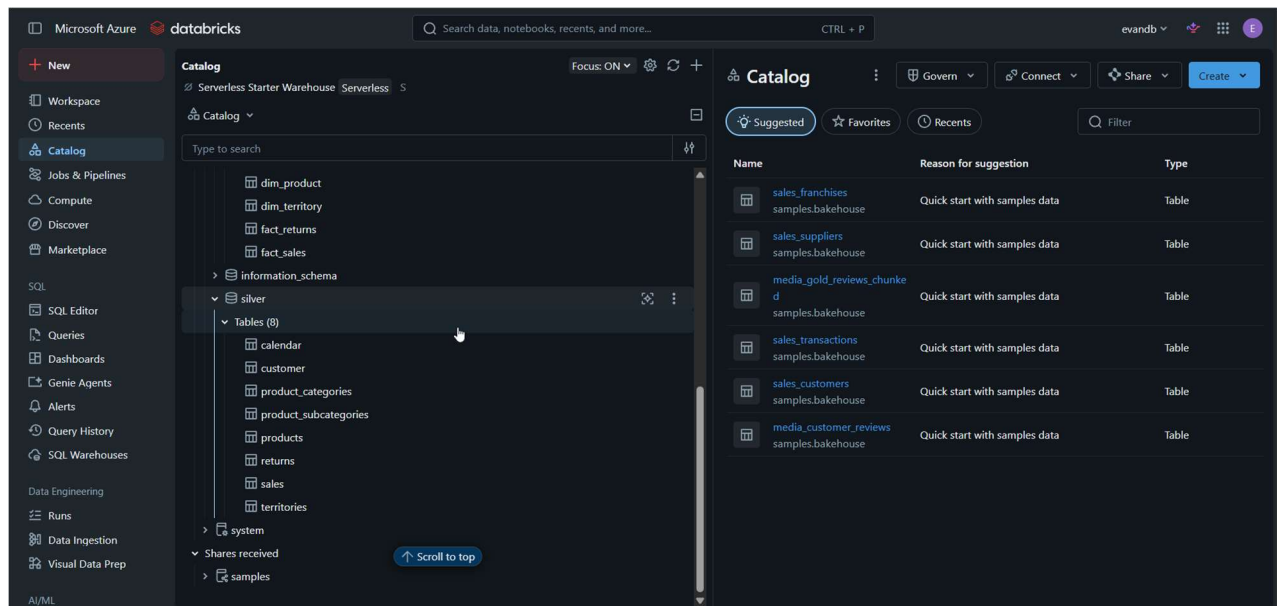


Figure 10. Unity Catalog Gold schema and registered Gold tables. (Source document page 4)

5. Bronze Layer

The Bronze layer contains the original AdventureWorks source files. The raw data is kept separate from the transformed layers so that the source can be reprocessed without altering the original inputs.

```
AdventureWorks_Calendar.csv
AdventureWorks_Customers.csv
AdventureWorks_Products.csv
AdventureWorks_Product_Categories.csv
AdventureWorks_Product_Subcategories.csv
AdventureWorks>Returns.csv
AdventureWorks_Sales*.csv
AdventureWorks_Territory.csv
```

Unity Catalog Bronze Tables

```
evandb_7405616343397725.bronze.calendar
evandb_7405616343397725.bronze.customers
evandb_7405616343397725.bronze.products
evandb_7405616343397725.bronze.product_categories
evandb_7405616343397725.bronze.product_subcategories
evandb_7405616343397725.bronze.returns
evandb_7405616343397725.bronze.sales
evandb_7405616343397725.bronze.territories
```

For the final project workflow, Bronze is treated as a read-only/raw layer. Transformations are performed downstream.

6. Silver Layer

The Silver layer contains cleaned and transformed Parquet datasets. The transformation stage prepares the data for analytical modeling.

Dataset	Main Transformation
Calendar	Converted Date to date type; added Year and Month.
Customer	Created Full_Name; calculated Age and AgeGroup.
Products	Separated product SKU and product-name information.
Sales	Converted StockDate to timestamp; standardized OrderNumber.
Categories / Subcategories	Prepared lookup data for Gold product modeling.
Returns	Prepared return data for Gold fact modeling.
Territories	Prepared territory attributes for Gold dimension modeling.

Silver Tables

```
calendar/  
customer/  
products/  
product_categories/  
product_subcategories/  
returns/  
sales/  
territories/
```

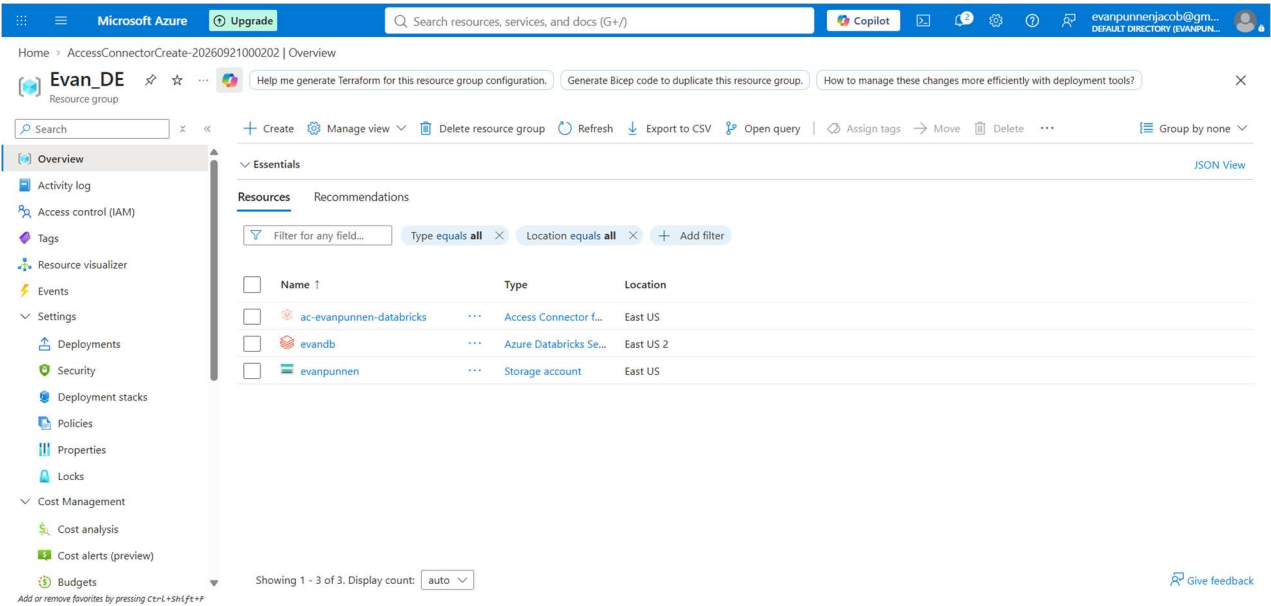


Figure 4. Databricks notebook showing Silver-layer transformation/write work. (Source document page 2)

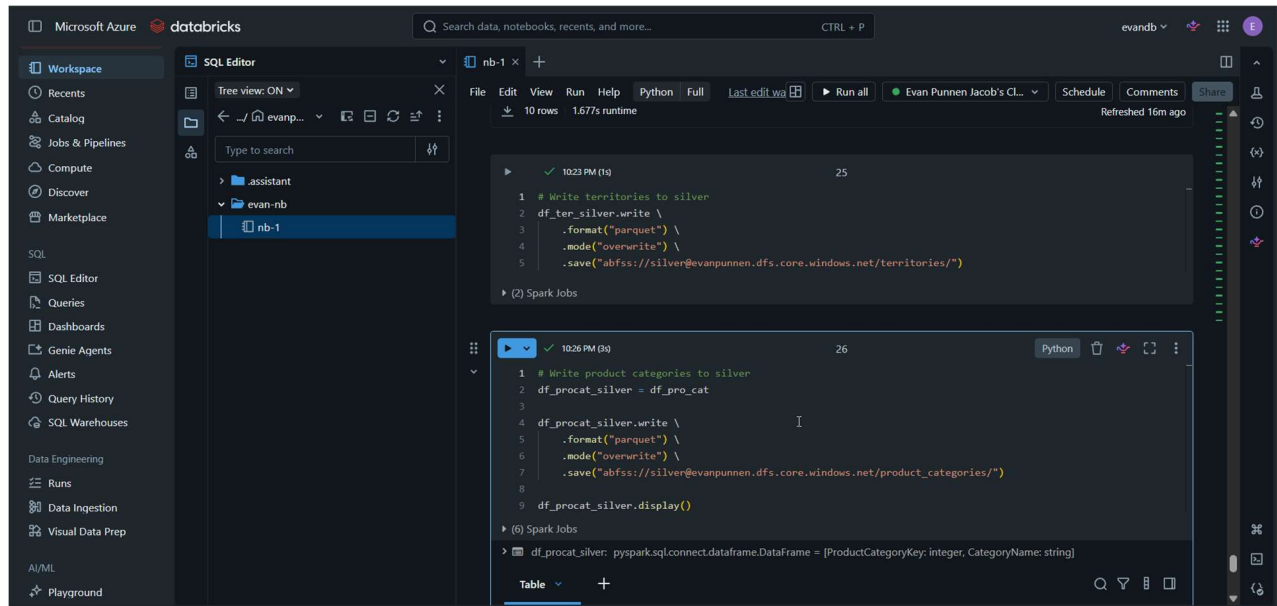


Figure 5. Databricks notebook showing additional Silver transformation/write cells. (Source document page 2)

7. Gold Layer and Star Schema

The Gold layer is the business-facing analytical layer. It uses a star schema with four dimensions and two fact tables.

Dimensions:
 dim_date
 dim_customer
 dim_product
 dim_territory

Facts:
 fact_sales
 fact_returns

7.1 dim_date

- DateKey
- Date
- Year
- Quarter
- Month
- MonthName
- YearMonth
- DayName
- WeekOfYear
- IsWeekend

7.2 dim_customer

- FullName

- Age
- AgeGroup

7.3 dim_product

- Product information
- SubcategoryName
- CategoryName
- ProductCost
- ProductPrice
- MarginPct

7.4 dim_territory

- TerritoryKey
- Region
- Country
- Continent

7.5 fact_sales

```

OrderNumber
OrderLineItem
OrderDateKey
StockDateKey
CustomerKey
ProductKey
TerritoryKey
OrderQuantity
UnitPrice
UnitCost
ExtendedRevenue
ExtendedCost
GrossProfit
OrderYear

```

Calculated measures:

```

ExtendedRevenue = OrderQuantity × ProductPrice
ExtendedCost = OrderQuantity × ProductCost
GrossProfit = ExtendedRevenue - ExtendedCost

```

7.6 fact_returns

```

ReturnDateKey
ProductKey
TerritoryKey
ReturnQuantity
ReturnValue

```

Calculated measure:

```

ReturnValue = ReturnQuantity × ProductPrice

```

8. Data Validation

Validation was performed before dashboard reporting. The checks focused on row counts, uniqueness, missing keys, null values, and referential integrity.

Validation	Result
Products	293 / 293 distinct
Customers	18,148 / 18,148 distinct
Territories	10 / 10 distinct
Dates	912 / 912 distinct
Missing Product Keys	0
Missing Customer Keys	0
Missing Territory Keys	0
Missing Order Dates	0

Validation principle:

Fact row count should remain consistent with the Silver source.
Every fact foreign key should exist in its corresponding dimension.
Orphan key count should be 0.

9. AI/BI Dashboard

The completed dashboard is named AdventureWorks Sales Dashboard. It uses Gold-layer data to present KPIs and business trends.

9.1 Dashboard Datasets

- KPI Summary
- Return Rate
- Revenue and Profit by Year
- Revenue by Category
- Return Rate by Category
- Top 10 Customers

9.2 KPI Query

```
SELECT
    SUM(ExtendedRevenue) AS Revenue,
    SUM(GrossProfit) AS GrossProfit,
    SUM(OrderQuantity) AS UnitsSold
FROM evandb_7405616343397725.gold.fact_sales
```

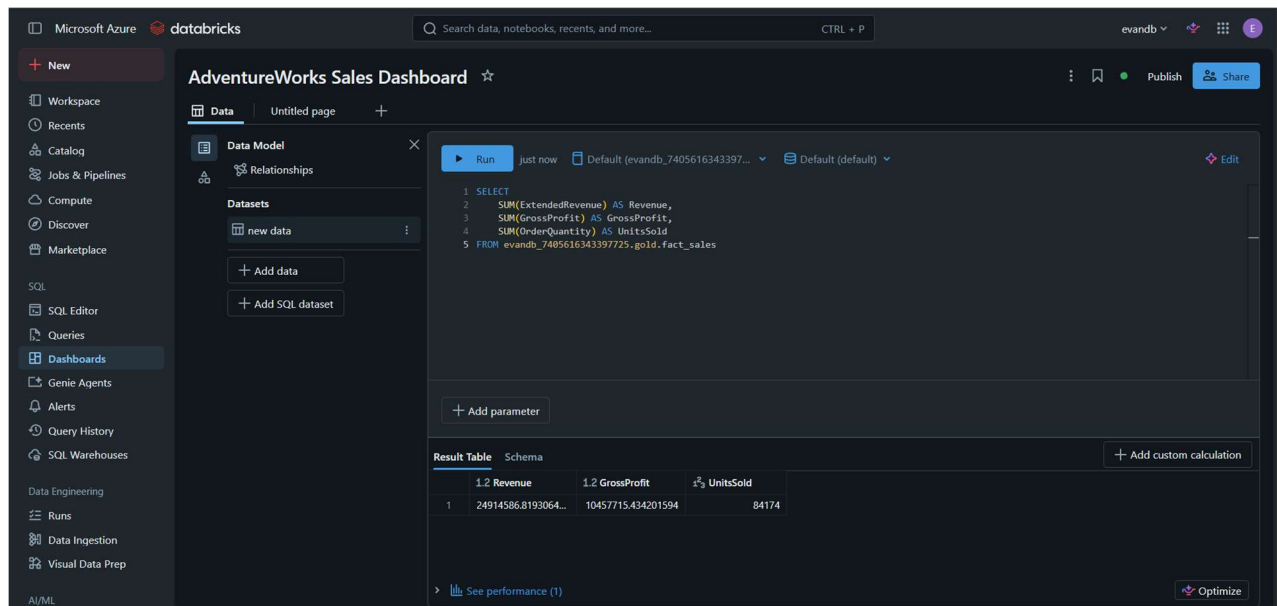


Figure 11. Dashboard dataset setup and KPI query. (Source document page 5)

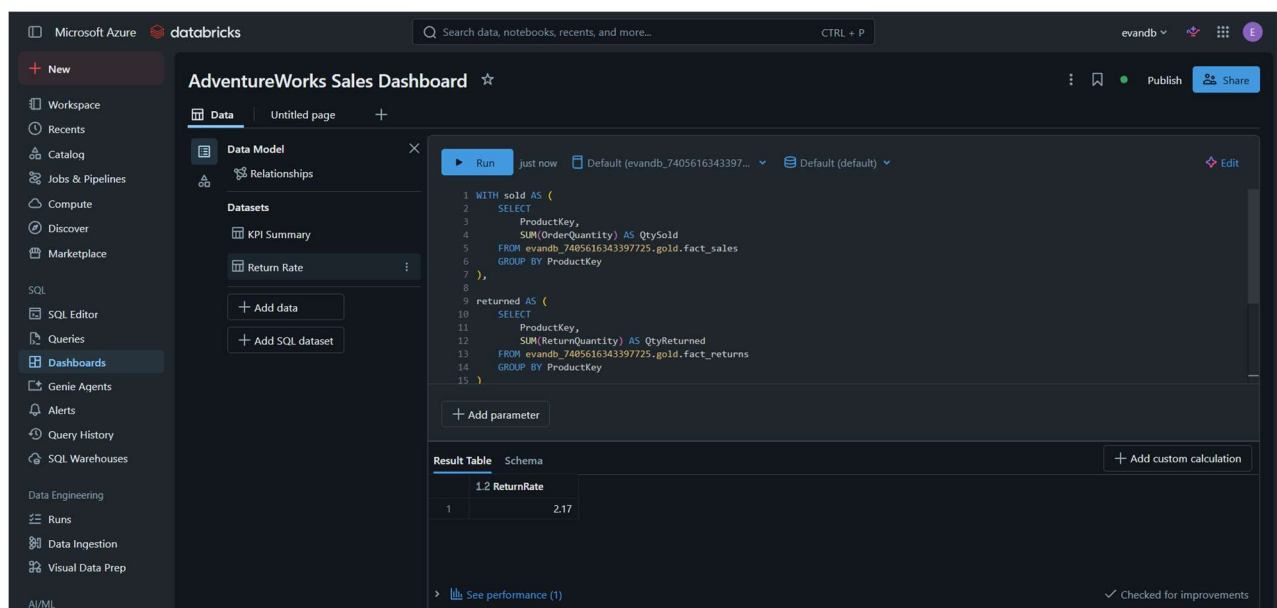


Figure 12. Dashboard dataset result for KPI analysis. (Source document page 5)

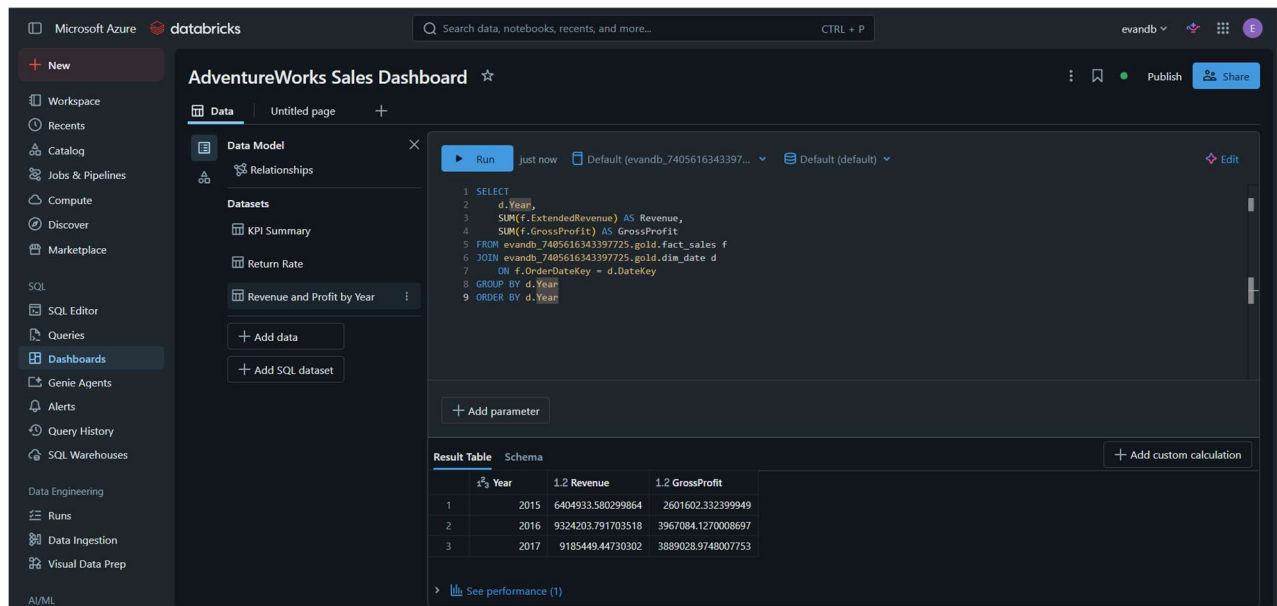


Figure 13. Additional dashboard dataset/query configuration. (Source document page 5)

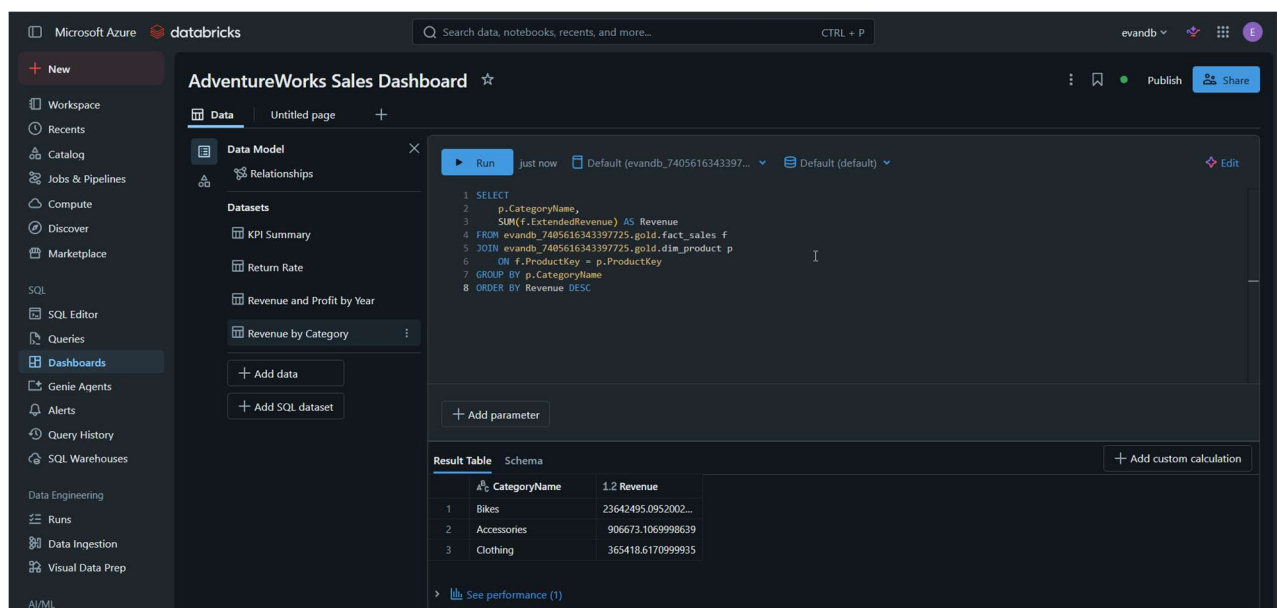


Figure 14. Dashboard dataset configuration for another business metric. (Source document page 6)

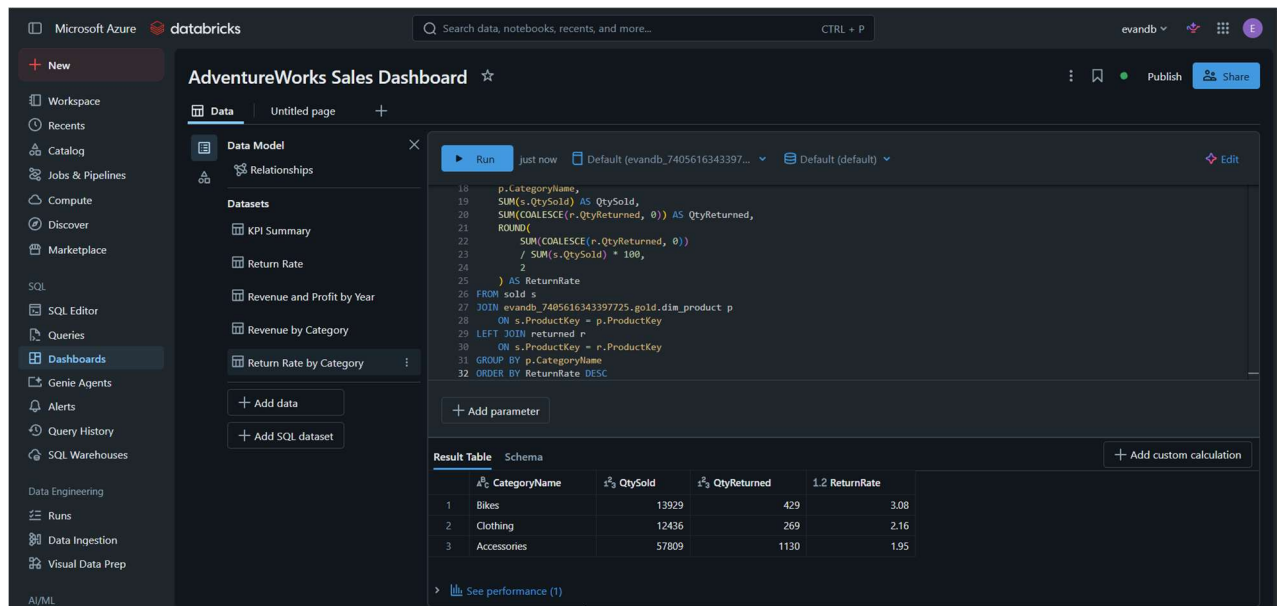


Figure 15. Dashboard result table for category/year analysis. (Source document page 6)

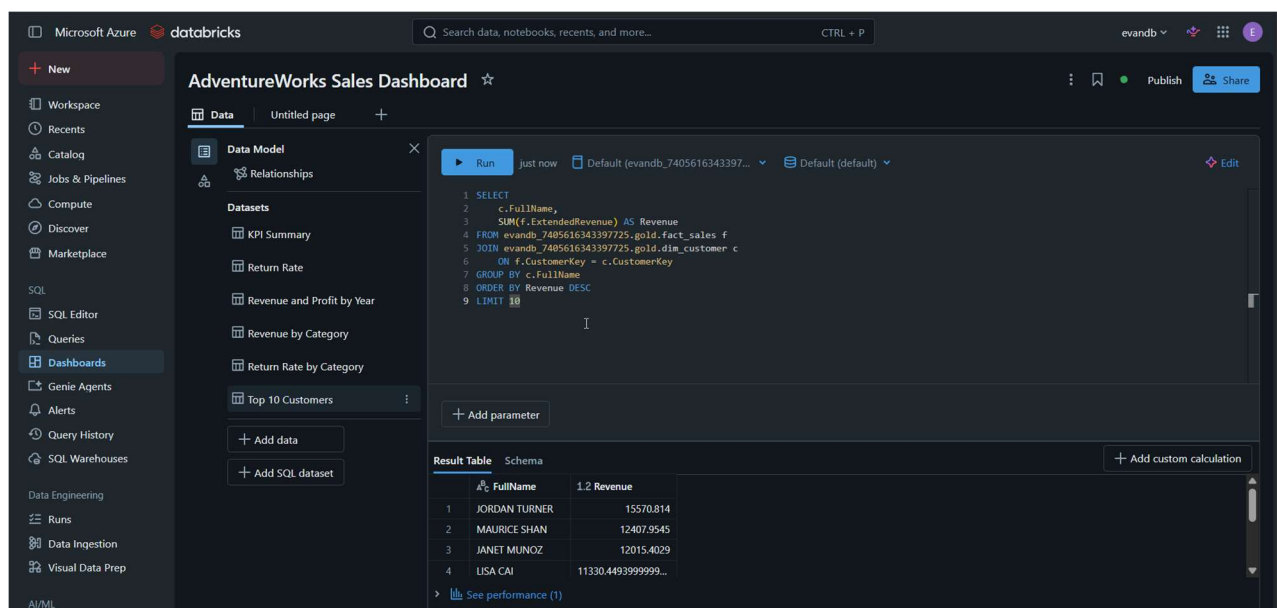


Figure 16. Dashboard result table for customer analysis. (Source document page 6)

9.3 Dashboard Visuals

#	Visual
1	Total Revenue
2	Gross Profit
3	Units Sold
4	Revenue by Year
5	Revenue by Category
6	Return Rate
7	Return Rate by Category
8	Top 10 Customers by Revenue

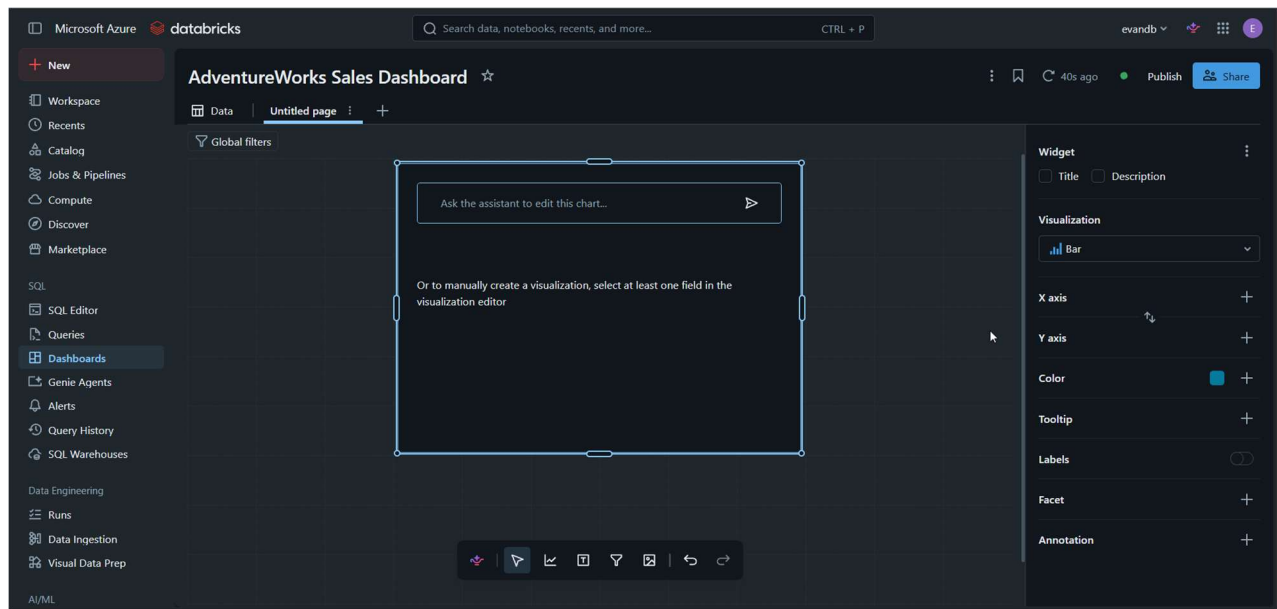


Figure 17. Initial KPI visual configuration. (Source document page 7)

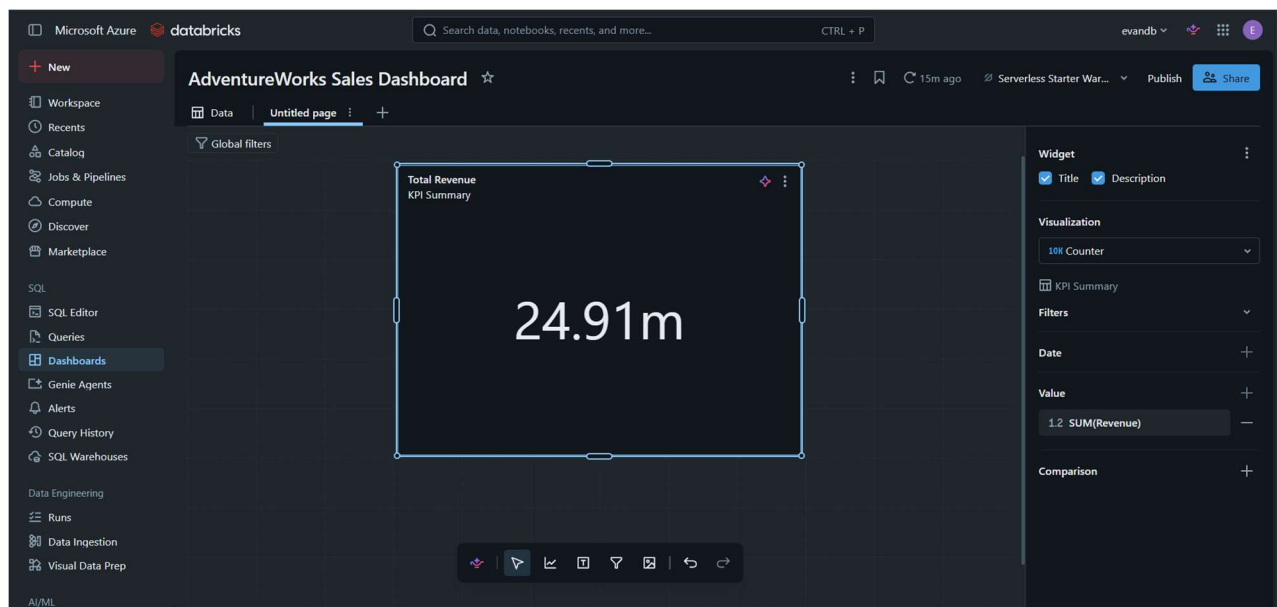


Figure 18. Total Revenue KPI visual showing approximately 24.91M. (Source document page 7)

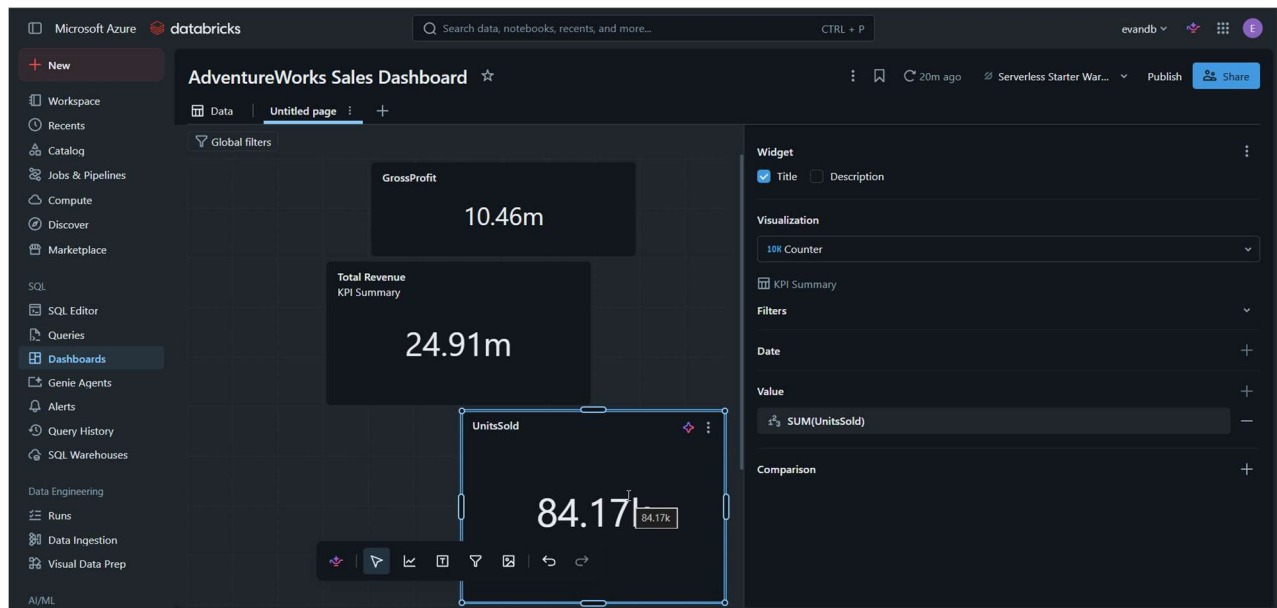


Figure 19. Dashboard KPI layout showing Revenue, Gross Profit and Units Sold. (Source document page 7)

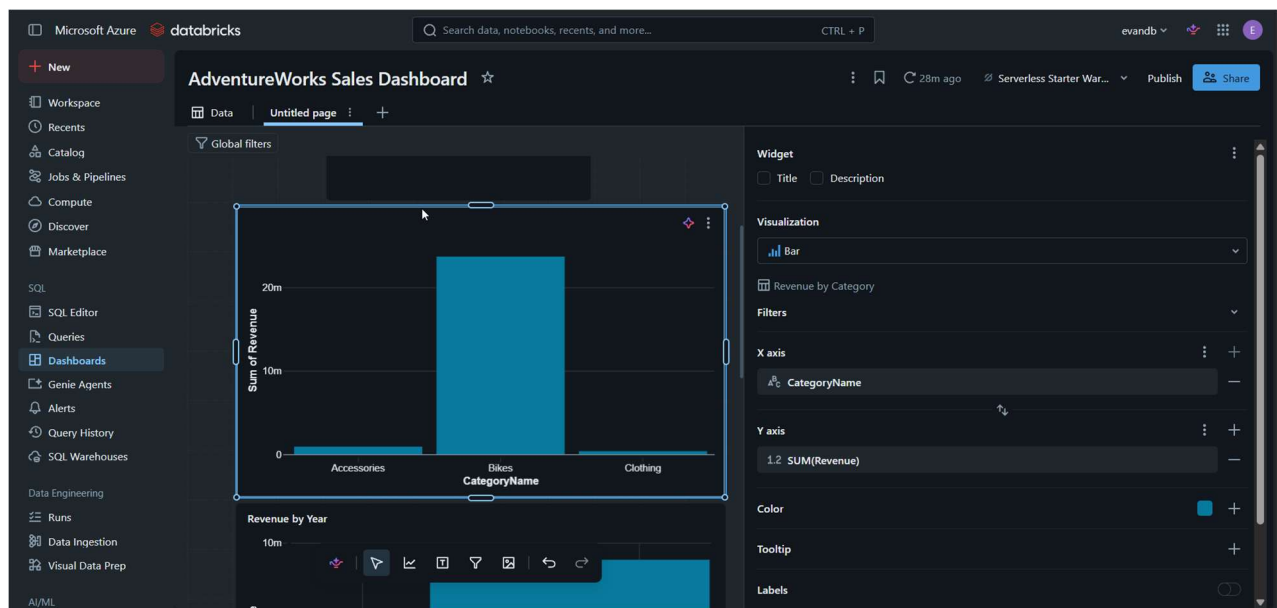


Figure 20. Revenue by Category visualization. (Source document page 8)

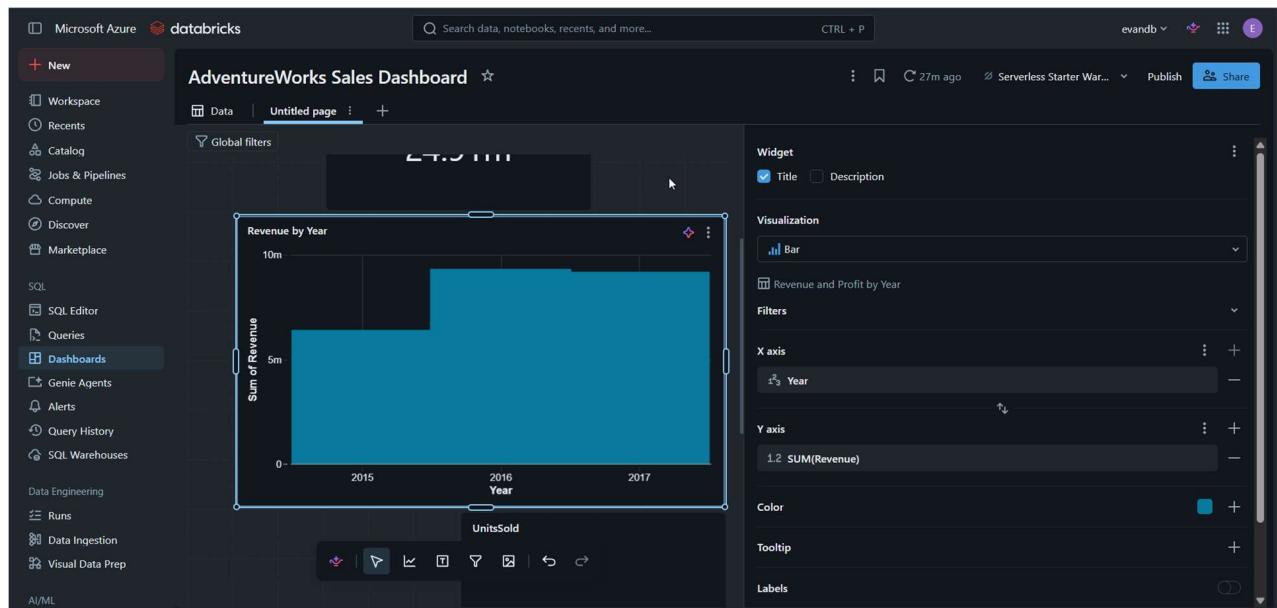


Figure 21. Revenue by Year visualization. (Source document page 8)

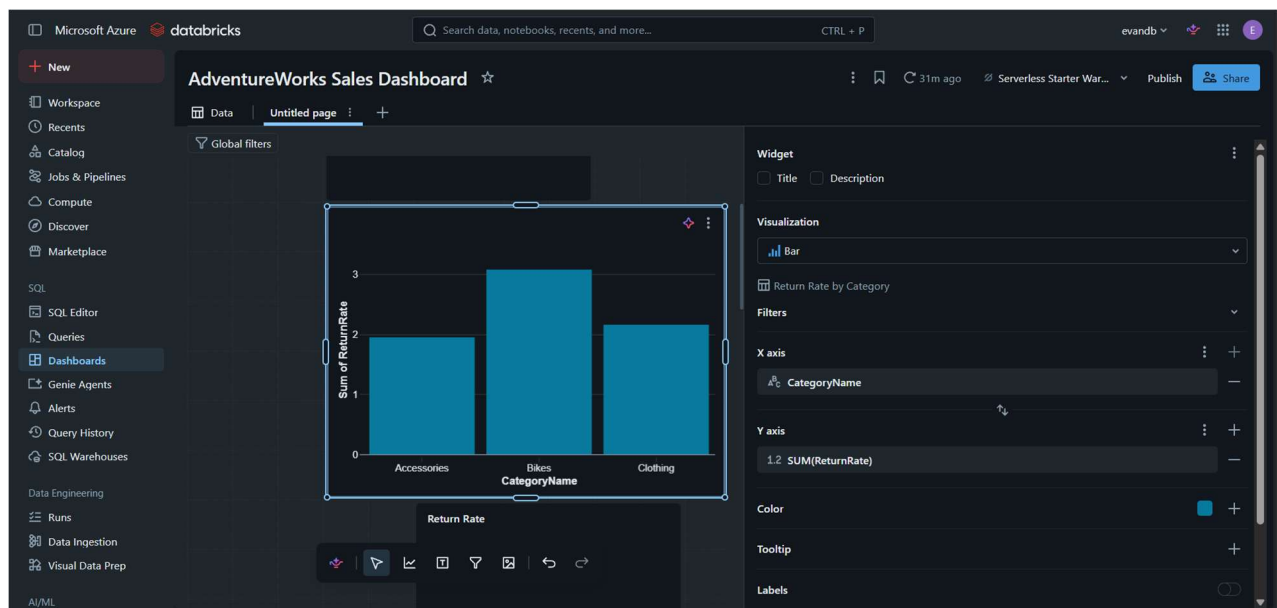


Figure 22. Category comparison visualization. (Source document page 8)

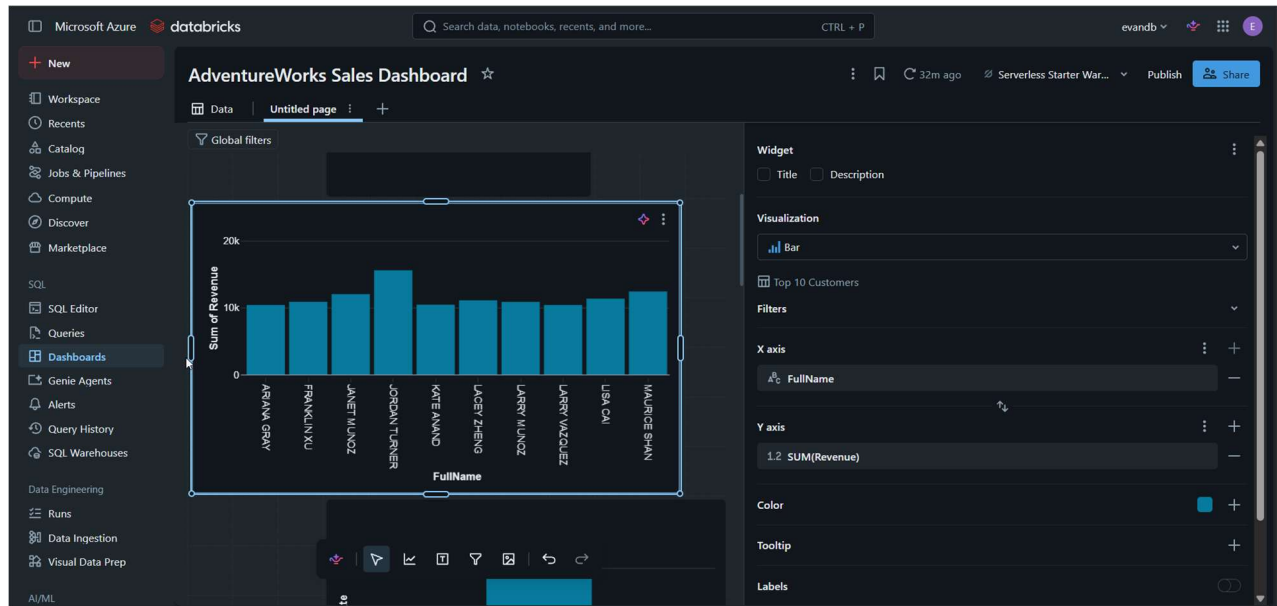


Figure 23. Top customer revenue visualization. (Source document page 9)

10. Dashboard Results

KPI	Value
Total Revenue	≈ 24.91M
Gross Profit	≈ 10.46M
Units Sold	84,174

The dashboard supports analysis of revenue, profit, units sold, yearly performance, category performance, returns, and top customers.

11. Job Automation — Intentionally Skipped

A Databricks Job named AdventureWorks_Medallion_Pipeline was created with the intended task flow Bronze_to_Silver → Silver_to_Gold. During automation testing, Unity Catalog path-overlap restrictions were encountered because the notebook still contained older direct ABFSS path reads/writes while Bronze/Silver paths were registered as Unity Catalog tables.

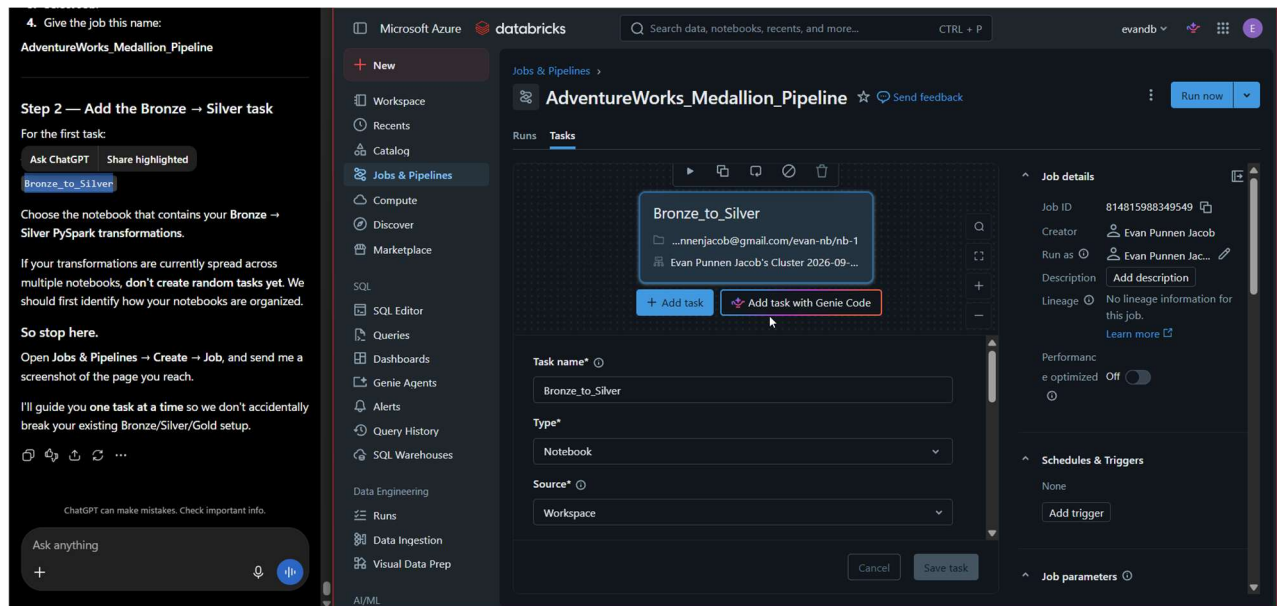


Figure 24. Databricks Job configuration showing the Medallion pipeline tasks. (Source document page 9)

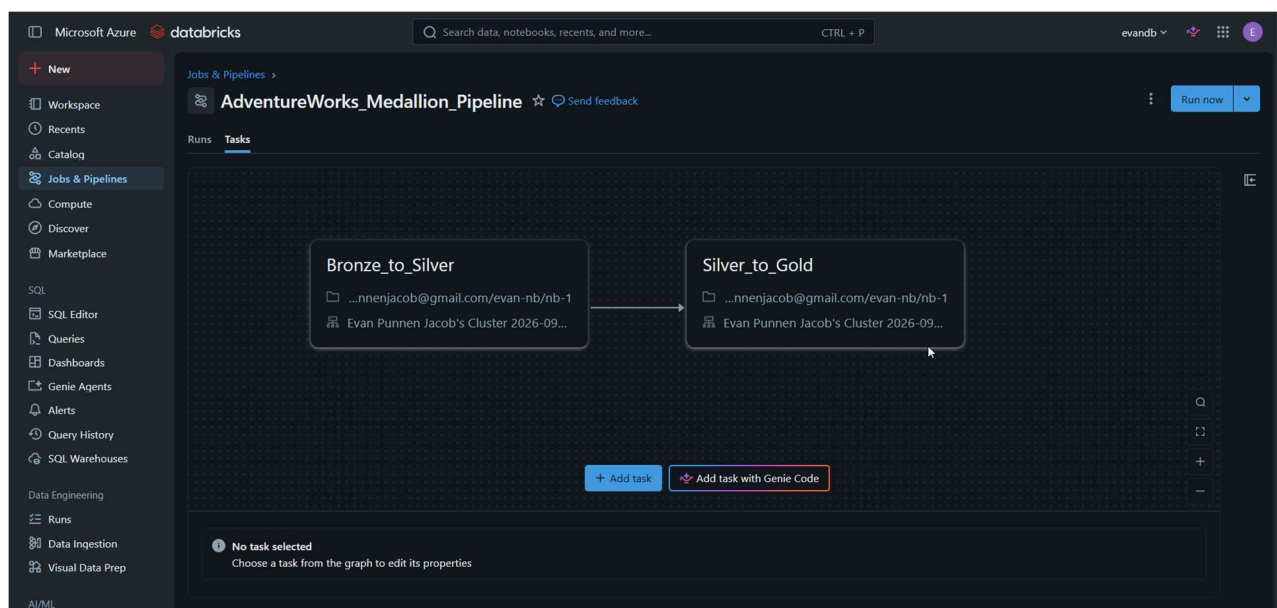


Figure 25. Databricks Job task dependency view: Bronze_to_Silver → Silver_to_Gold. (Source document page 9)

For project completion, automation was intentionally left out of the final execution path. The completed data layers, Unity Catalog setup, validation, and dashboard remain the deliverables.

12. Final Project Structure

AdventureWorks-Sales-Analytics/

- README.md
- notebooks/
 - Bronze_to_Silver



13. Reproduction Checklist

1. Create or identify the ADLS Gen2 storage account.
2. Create bronze, silver, and gold containers.
3. Upload the AdventureWorks CSV source files to Bronze.
4. Configure Databricks access to ADLS through the Azure managed identity / Access Connector.
5. Create the Unity Catalog storage credential.
6. Create the Bronze, Silver, and Gold external locations.
7. Create the Unity Catalog catalog and bronze, silver, and gold schemas.
8. Register Bronze source tables.
9. Read Bronze data with Spark/Databricks.
10. Apply Silver transformations and write the Silver datasets.
11. Read Silver tables for Gold modeling.
12. Build the four dimension tables.
13. Build fact_sales and fact_returns.
14. Validate row counts, nulls, duplicates, and orphan keys.
15. Register/verify Gold tables in Unity Catalog.
16. Create dashboard datasets.
17. Create KPI and analytical visuals.
18. Publish the AI/BI dashboard.
19. Keep the optional Job automation disabled/skipped for this completed version.

14. Security and Operational Notes

- Do not commit Azure client secrets, access keys, tokens, passwords, or connection strings to GitHub.
- Use managed identity / Access Connector and Unity Catalog for governed storage access.
- Keep Bronze data unchanged and treat it as the raw source layer.
- Use controlled overwrite/append strategies appropriate to the table design.

- Before enabling Job automation, remove legacy direct ABFSS reads/writes that conflict with Unity Catalog external tables.

15. Final Completion Status

Component	Status	Notes
Bronze Layer	COMPLETED	Raw AdventureWorks source data available.
Silver Layer	COMPLETED	Cleaned/transformed datasets created.
Gold Layer	COMPLETED	Four dimensions and two facts.
Unity Catalog	COMPLETED	Catalog, schemas, credentials and external locations configured.
Data Validation	COMPLETED	Counts, uniqueness, missing keys and integrity checks performed.
AI/BI Dashboard	COMPLETED	KPIs and business visuals created and published.
Job Automation	SKIPPED	Optional automation was not required for final project completion.

PROJECT DELIVERABLE STATUS: COMPLETED

Appendix A — Screenshot Evidence

The screenshots below are preserved as implementation evidence from the supplied project document. They are grouped by the implementation sequence and referenced throughout the report.

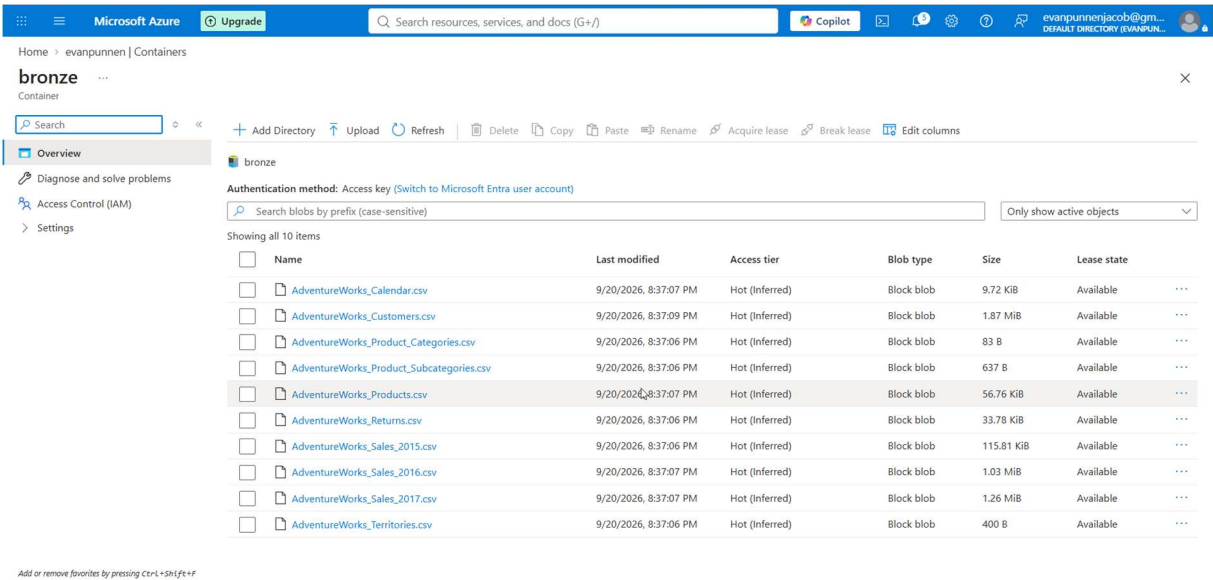


Figure 1. Implementation evidence screenshot.

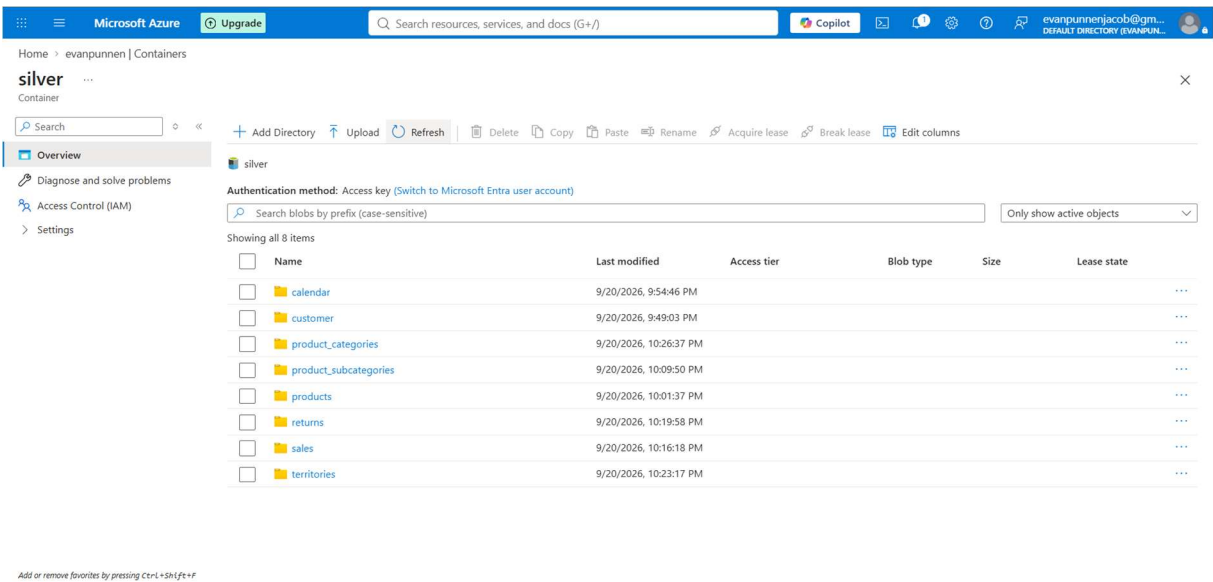


Figure 2. Implementation evidence screenshot.

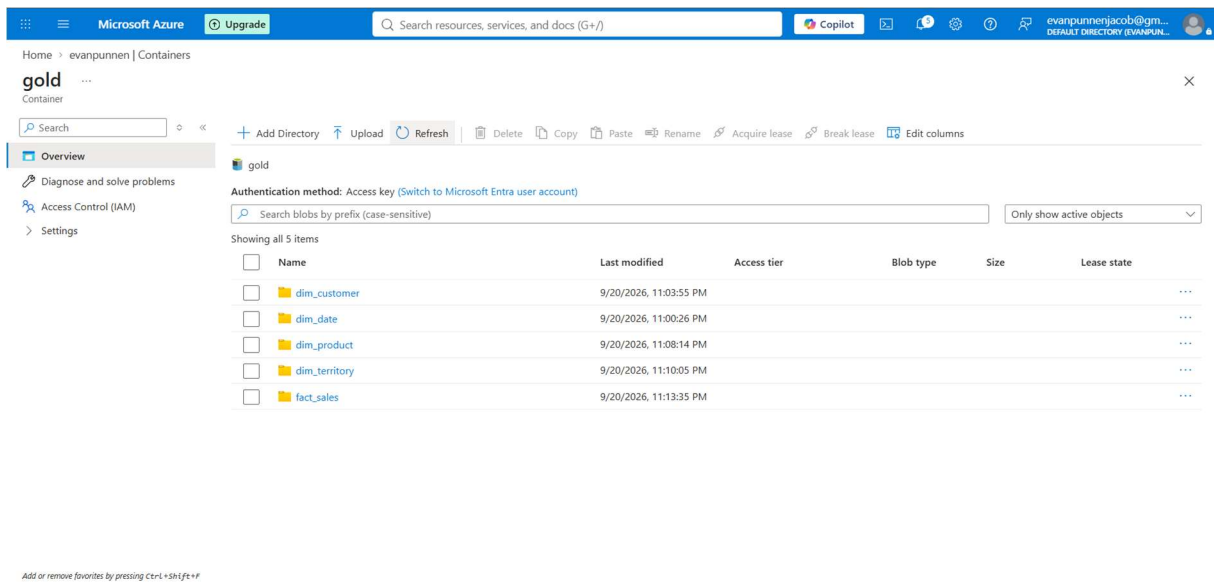


Figure 3. Implementation evidence screenshot.

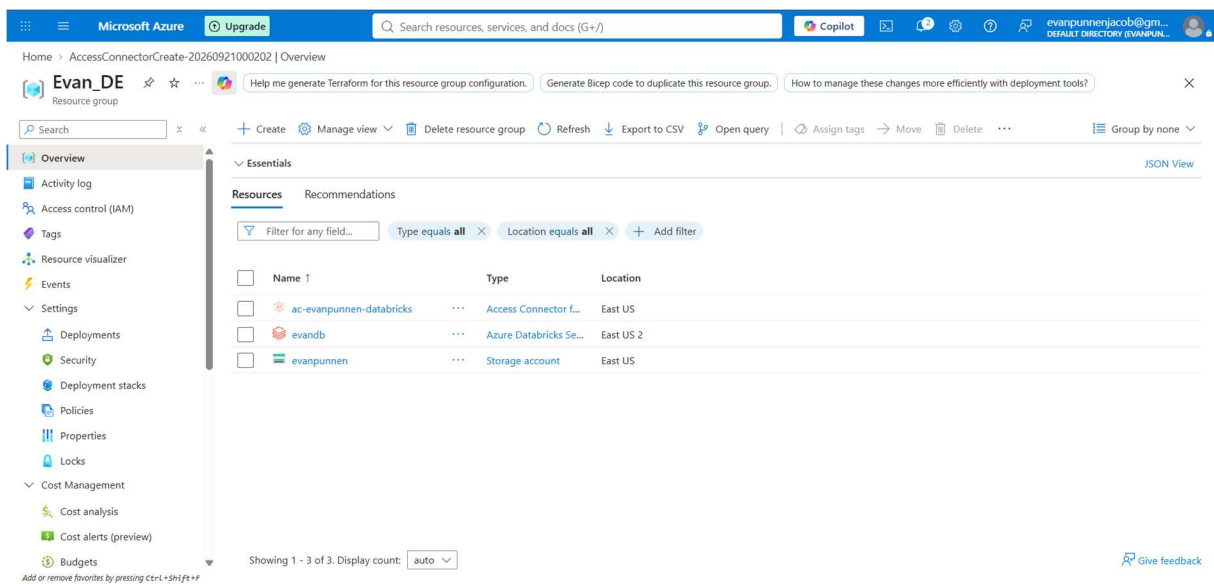


Figure 4. Implementation evidence screenshot.

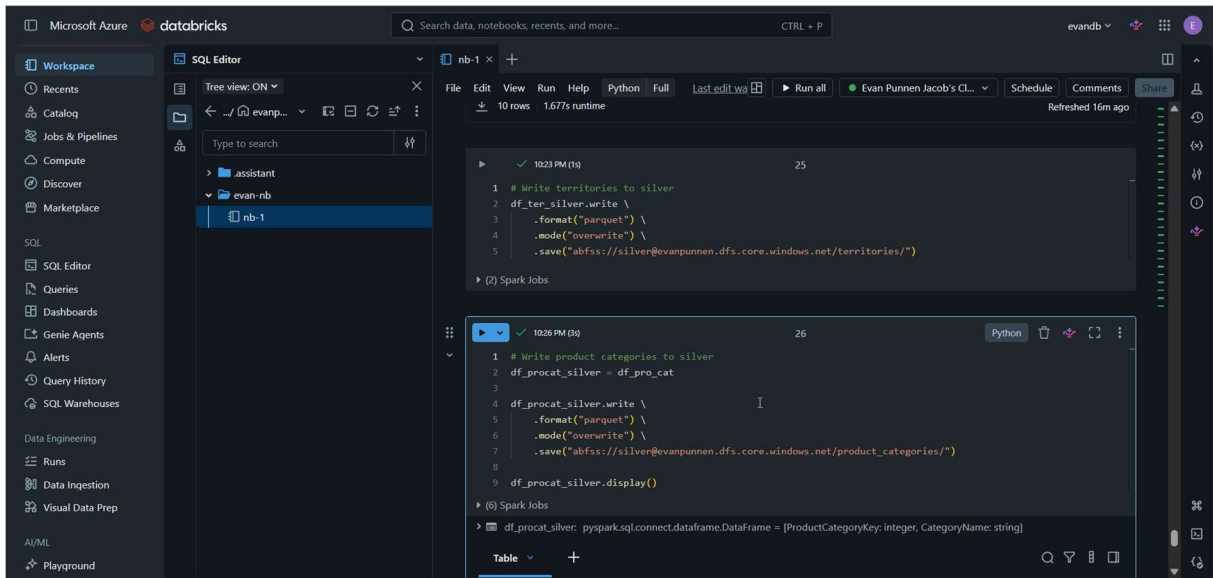


Figure 5. Implementation evidence screenshot.

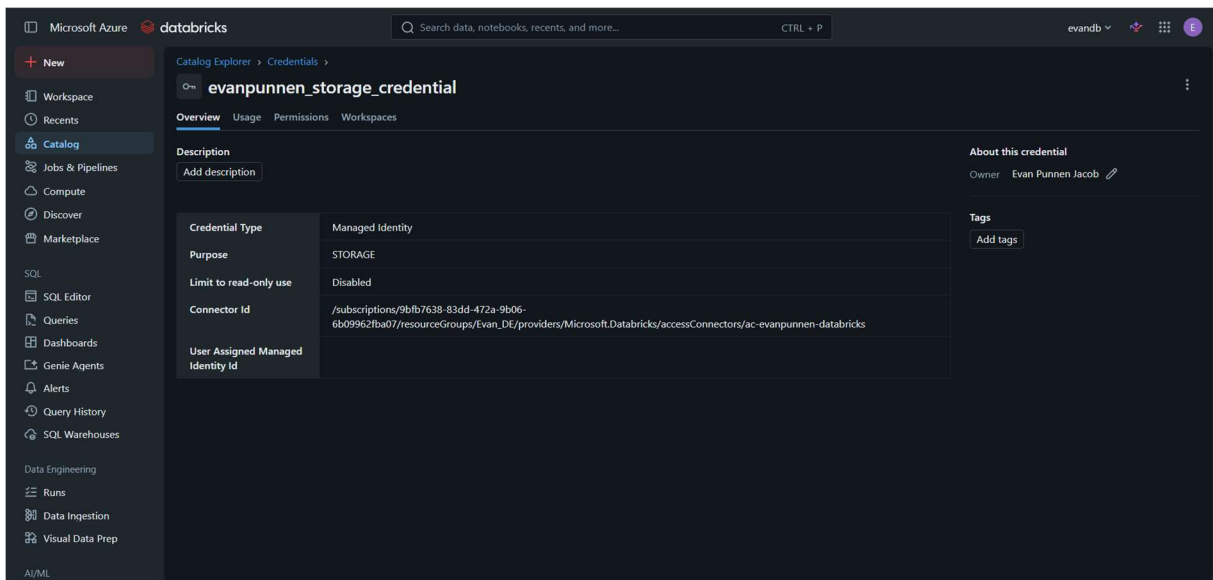


Figure 6. Implementation evidence screenshot.

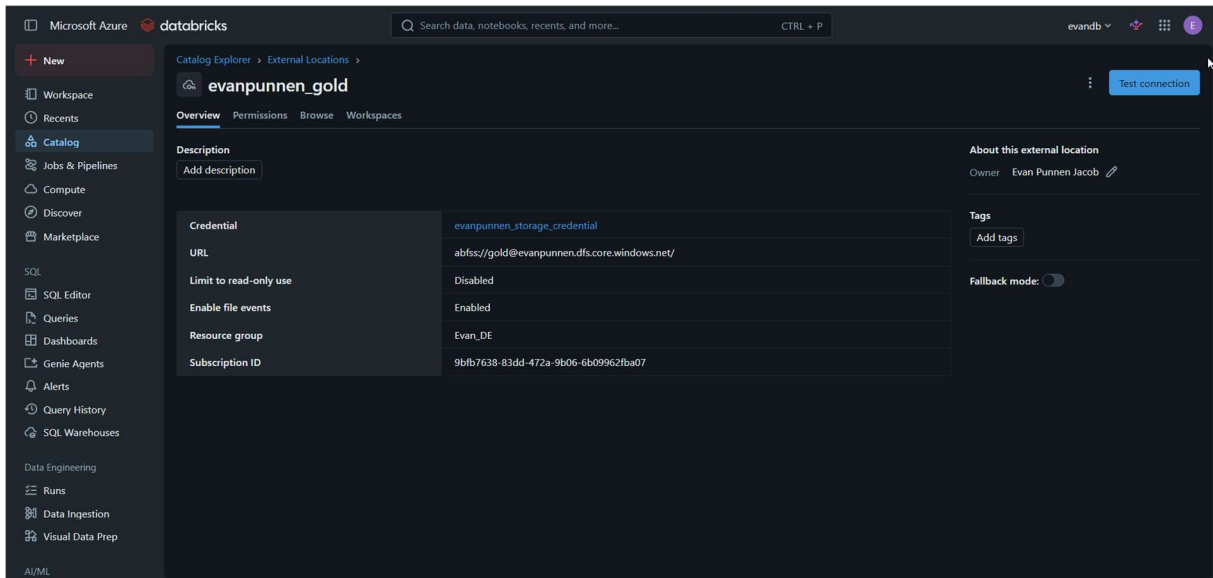


Figure 7. Implementation evidence screenshot.

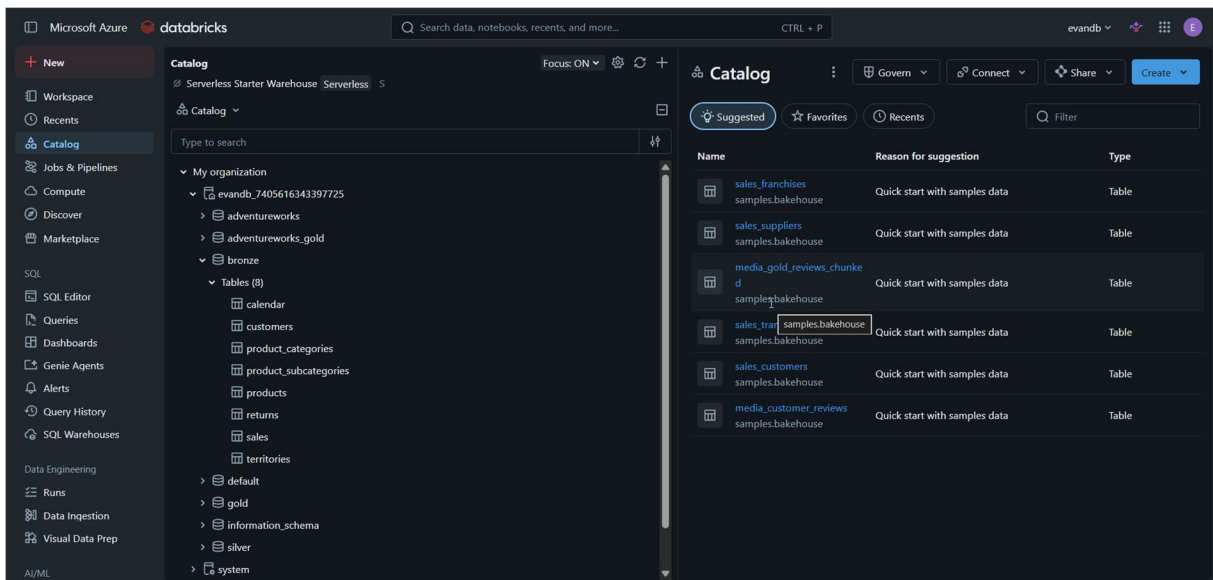


Figure 8. Implementation evidence screenshot.

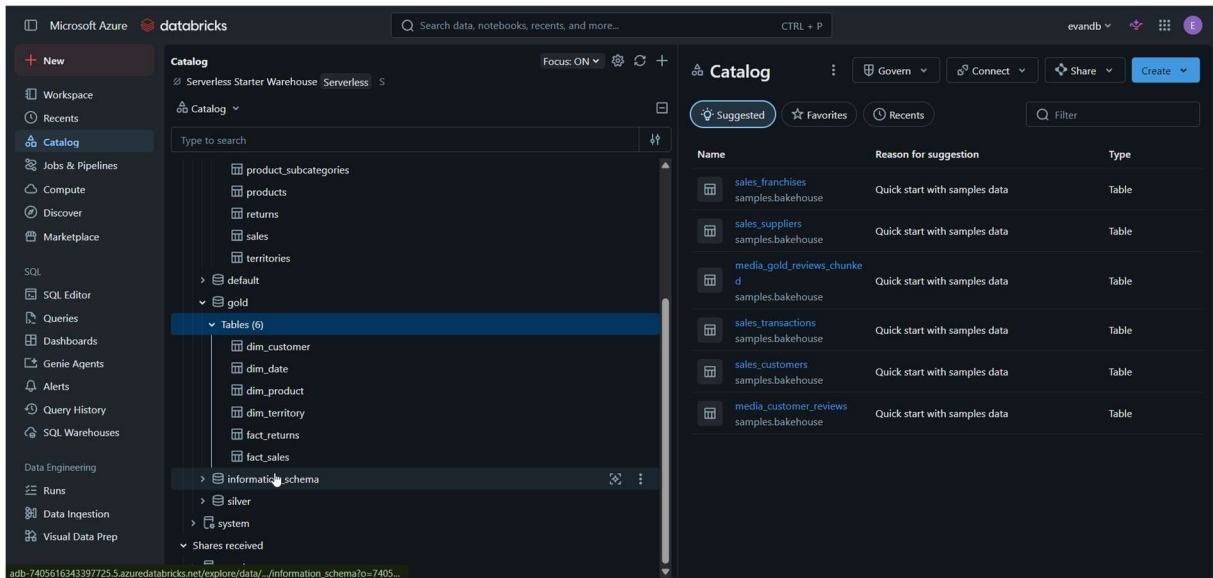


Figure 9. Implementation evidence screenshot.

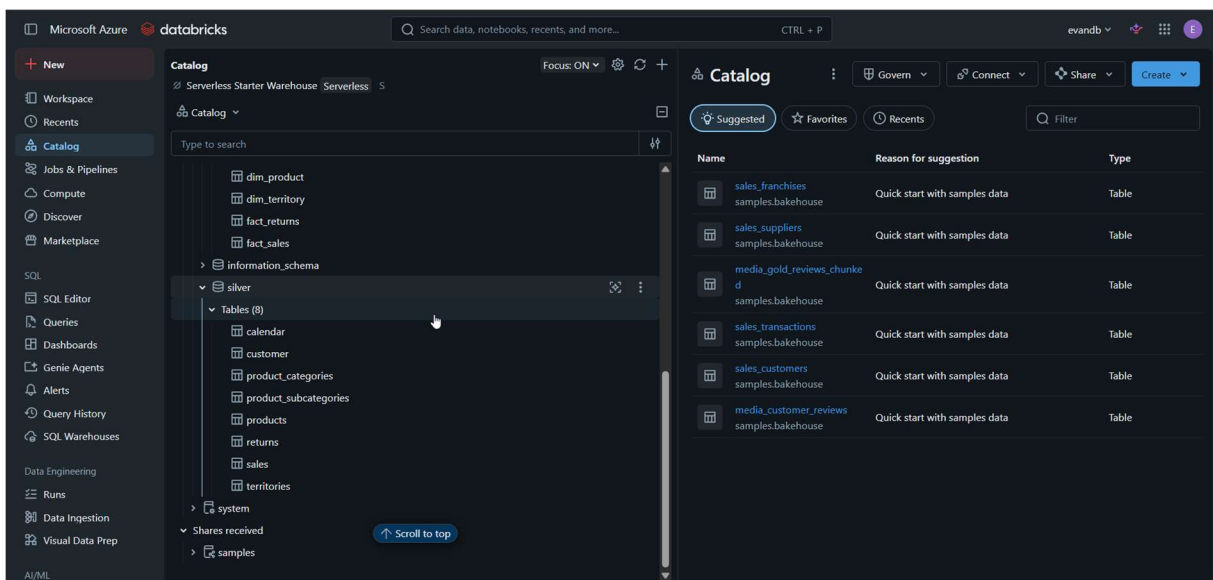


Figure 10. Implementation evidence screenshot.

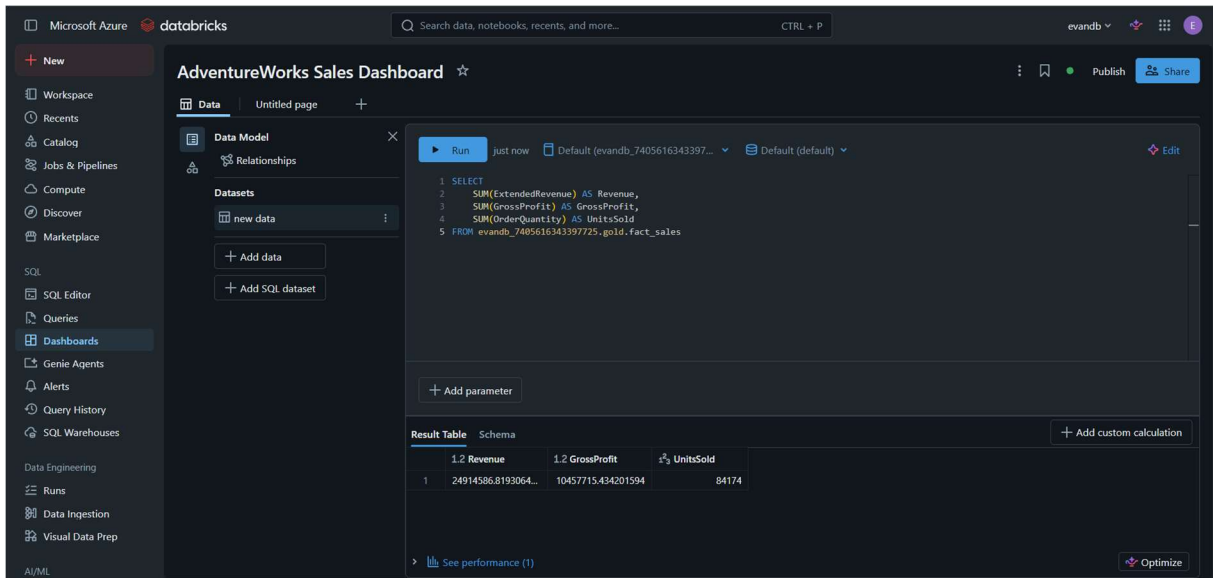


Figure 11. Implementation evidence screenshot.

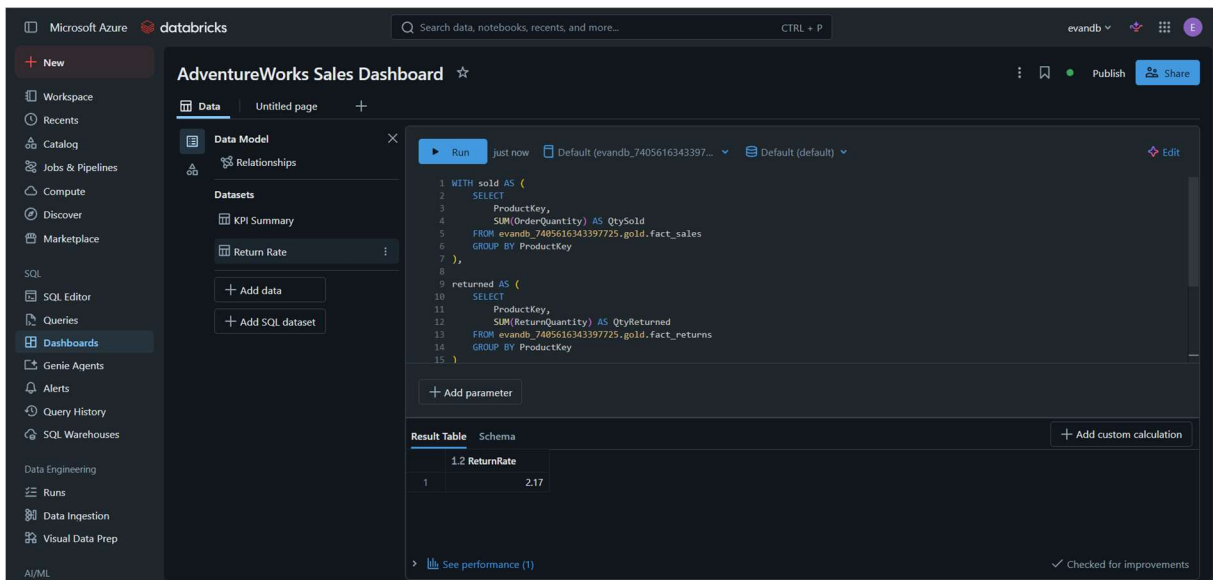


Figure 12. Implementation evidence screenshot.

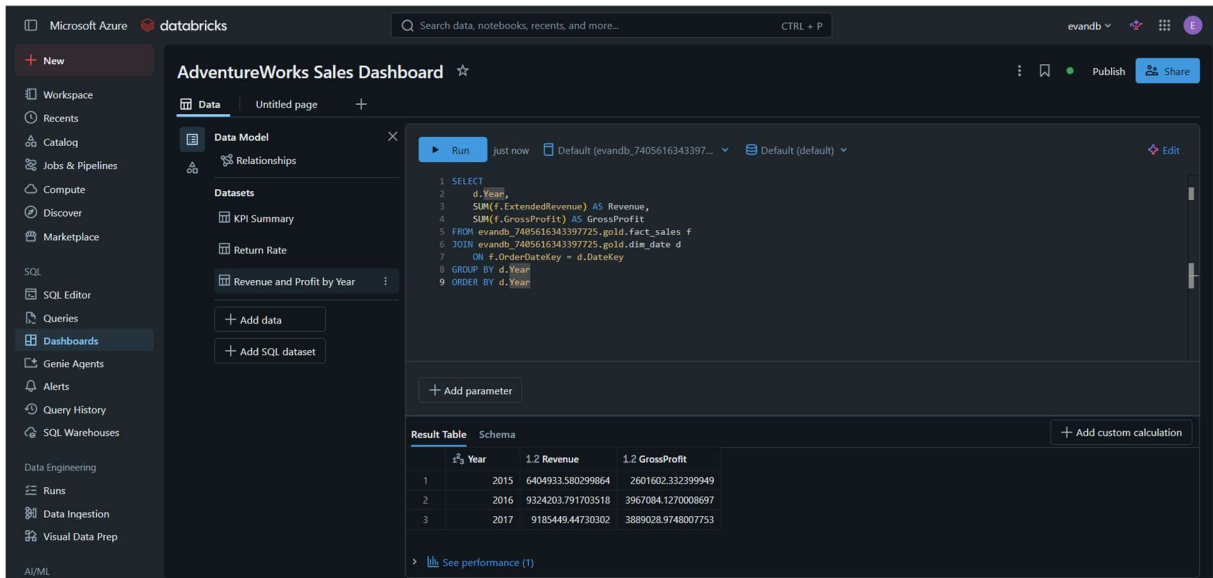


Figure 13. Implementation evidence screenshot.

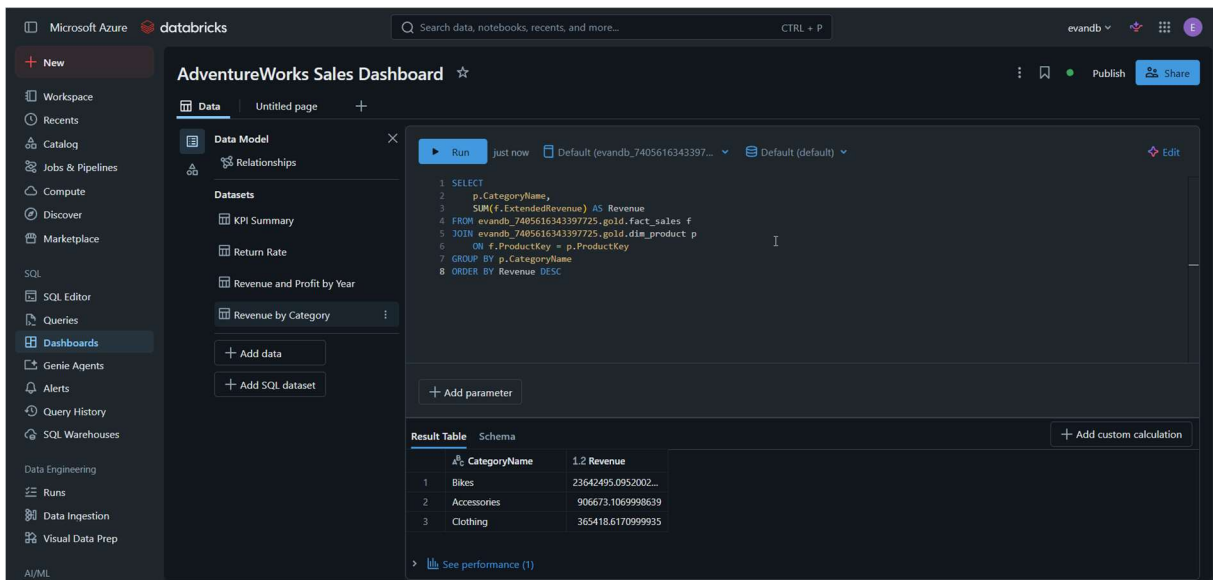


Figure 14. Implementation evidence screenshot.

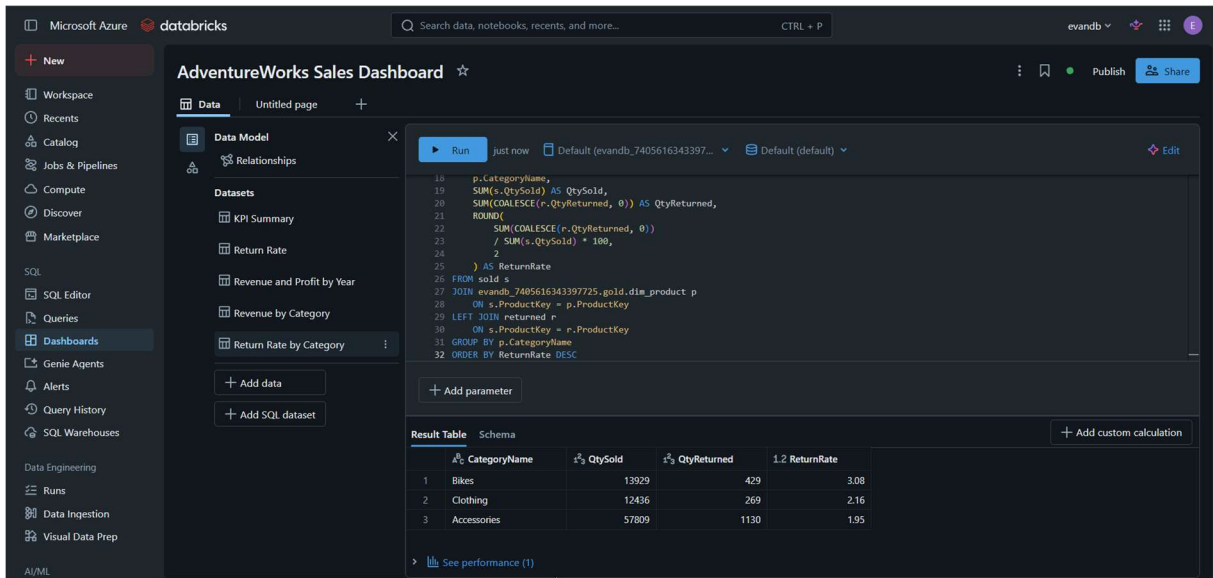


Figure 15. Implementation evidence screenshot.

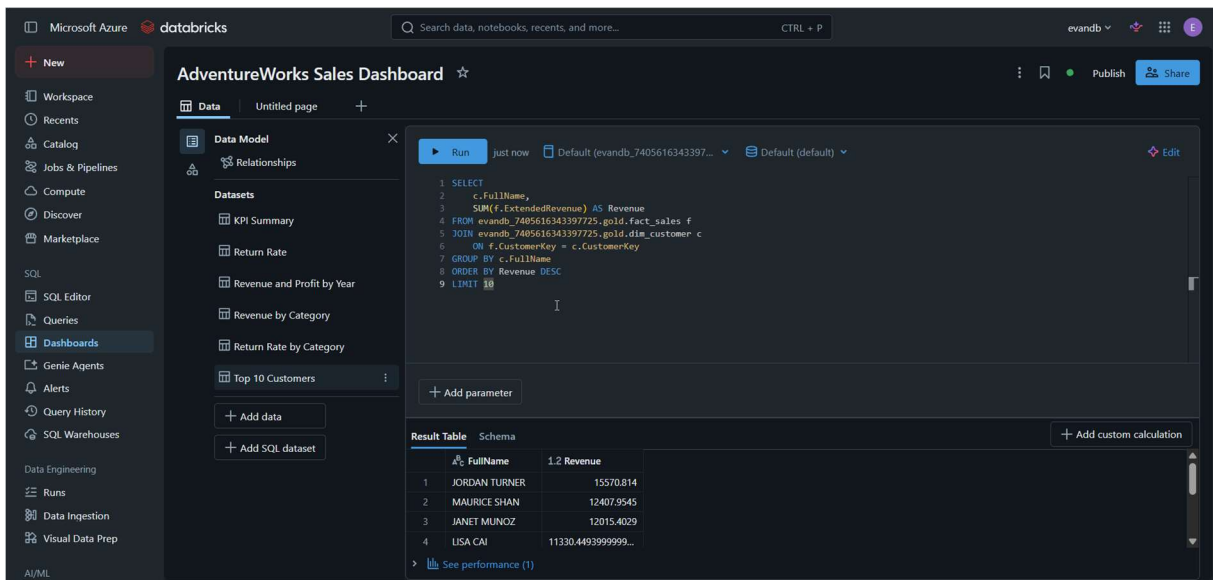


Figure 16. Implementation evidence screenshot.

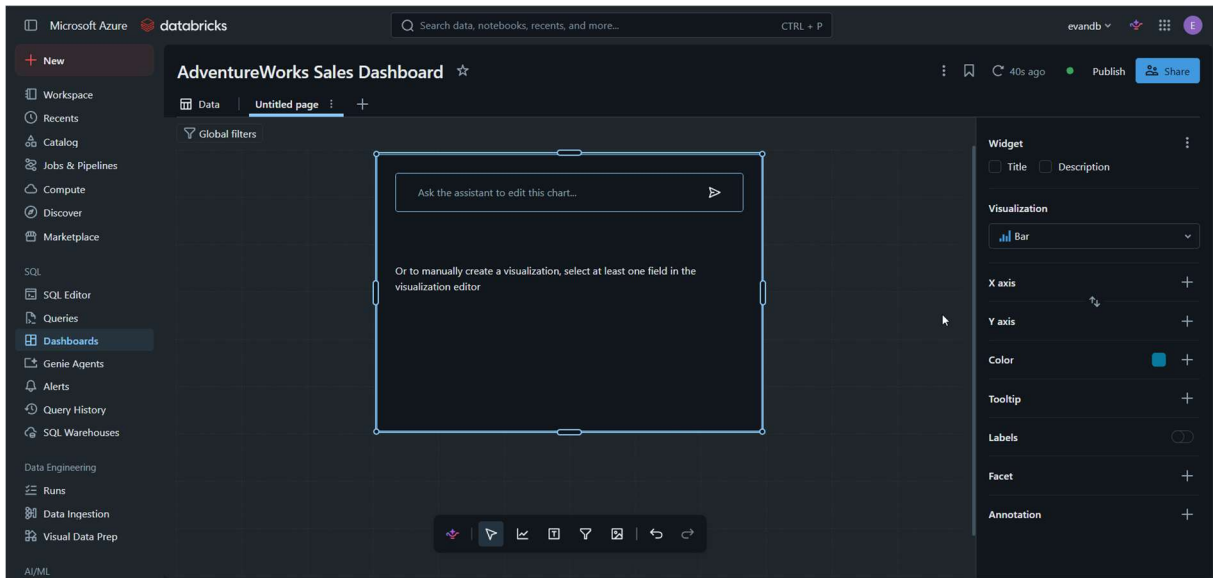


Figure 17. Implementation evidence screenshot.

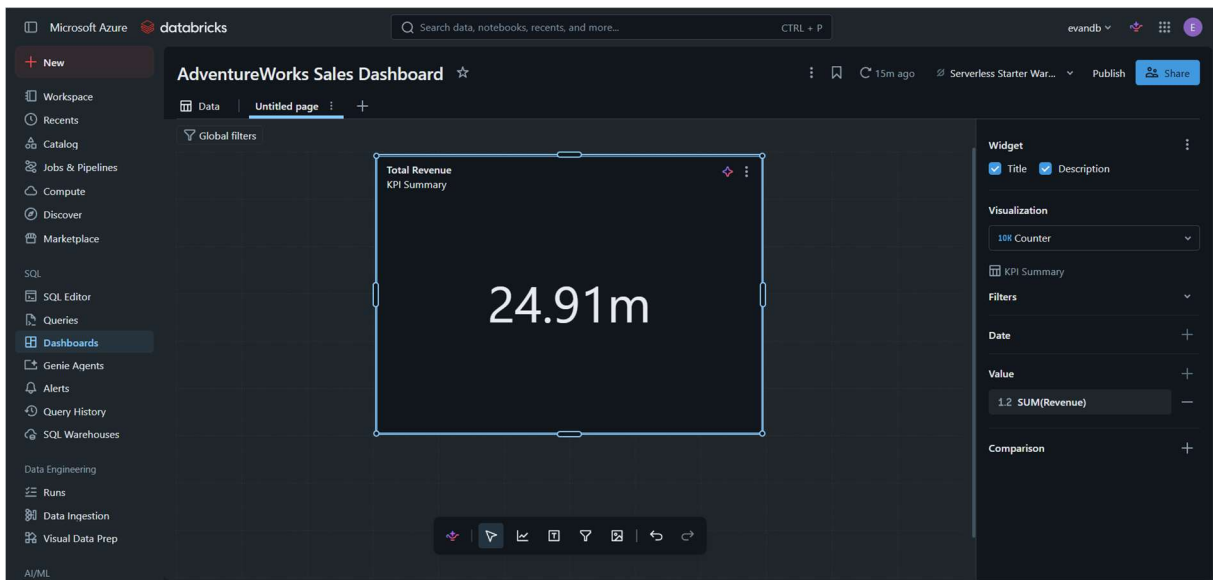


Figure 18. Implementation evidence screenshot.

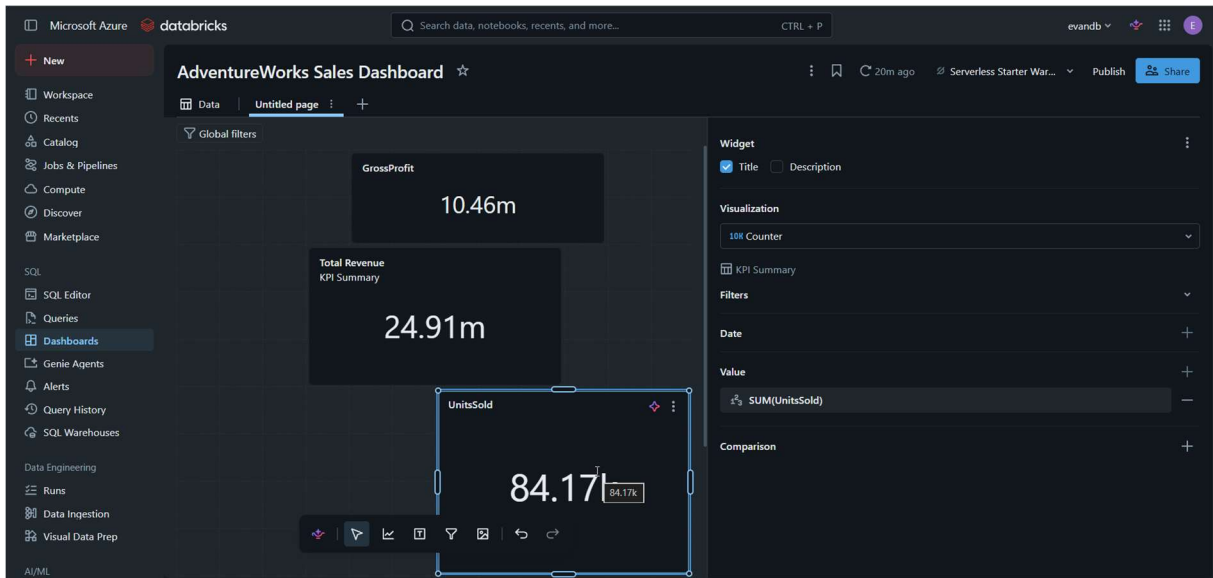


Figure 19. Implementation evidence screenshot.

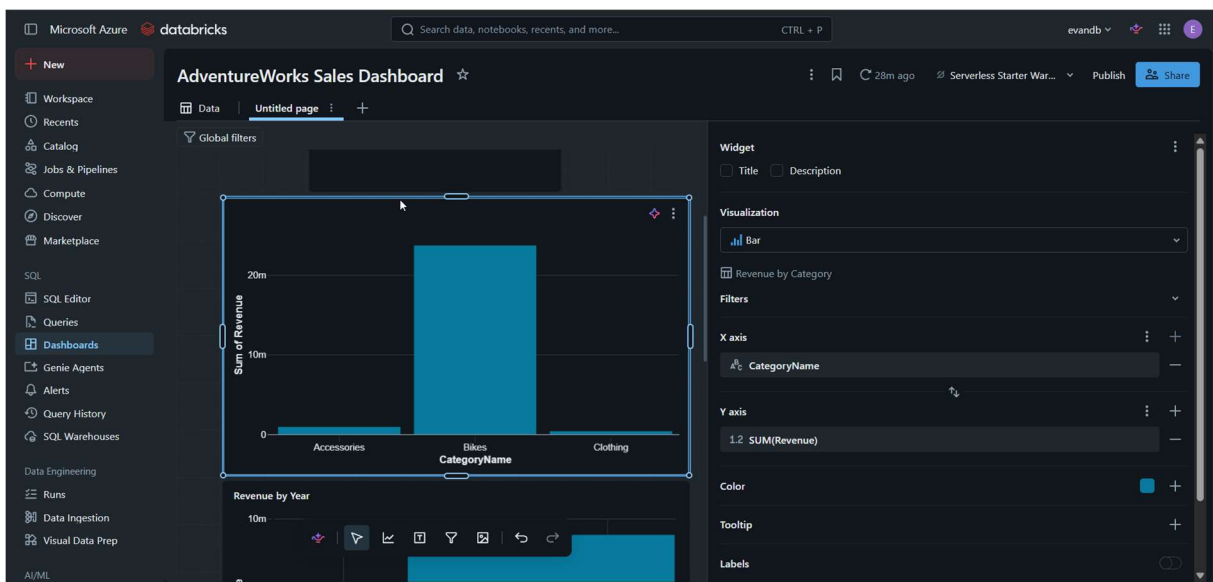


Figure 20. Implementation evidence screenshot.

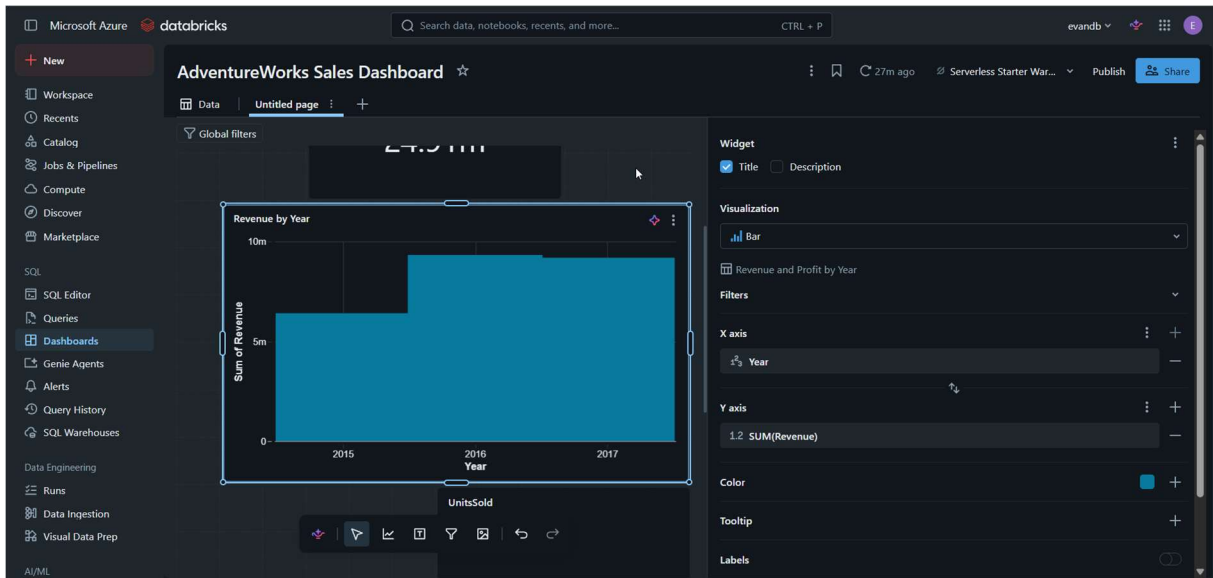


Figure 21. Implementation evidence screenshot.

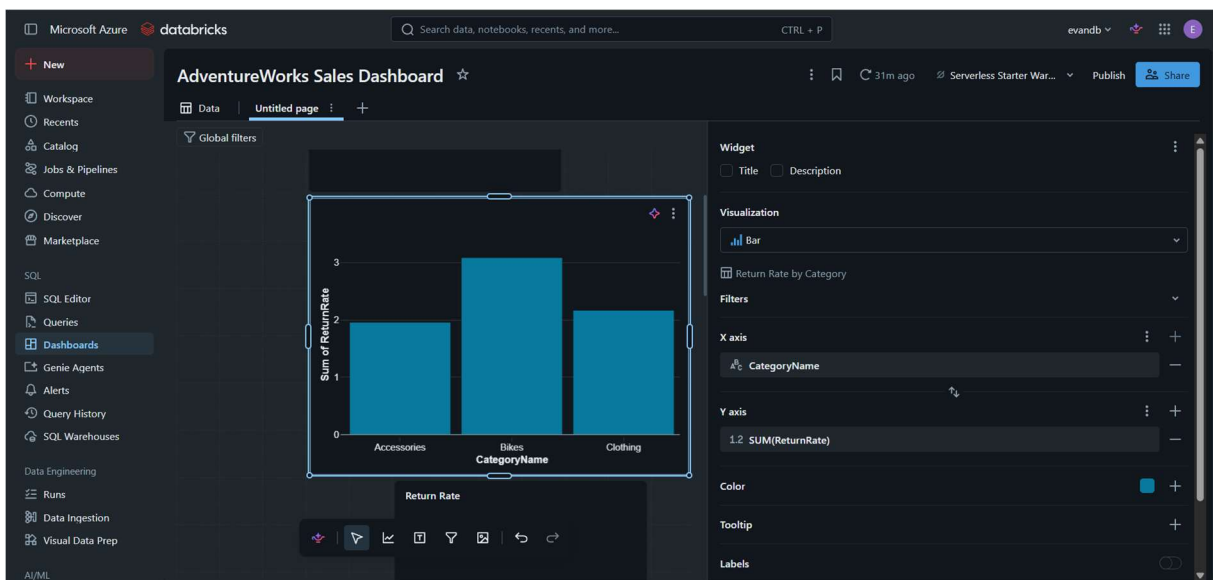


Figure 22. Implementation evidence screenshot.

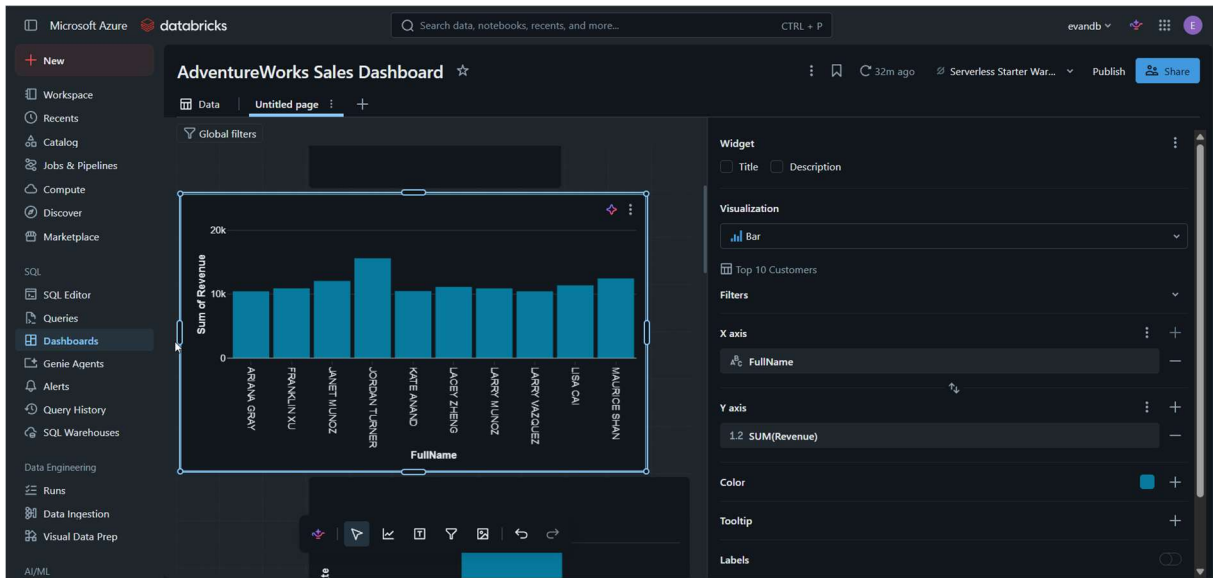


Figure 23. Implementation evidence screenshot.

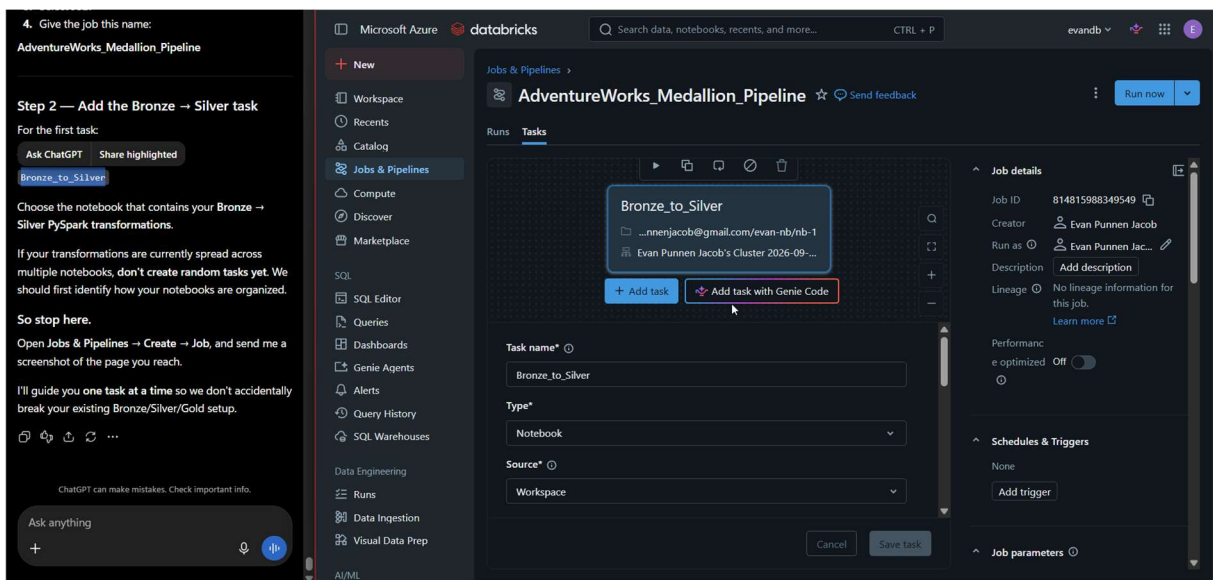


Figure 24. Implementation evidence screenshot.

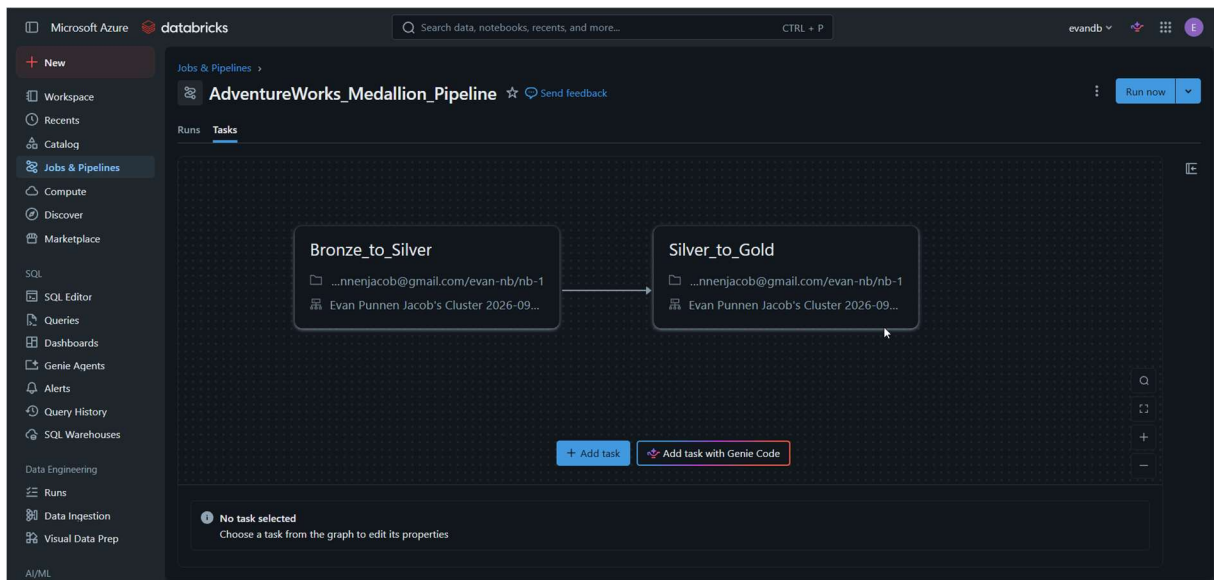


Figure 25. Implementation evidence screenshot.

Appendix B — Notebook Code

The notebook code is maintained as a separate project file. The supplied screenshot report did not contain a machine-readable notebook export, so this appendix intentionally does not invent or reproduce notebook code. When the notebook/code file is provided, it can be attached alongside this report or incorporated into a revised Appendix B.

Core Unity Catalog Bronze loading pattern used in the final troubleshooting workflow:

```
df_cal = spark.table("evandb_7405616343397725.bronze.calendar")
df_cus = spark.table("evandb_7405616343397725.bronze.customers")
df_pro = spark.table("evandb_7405616343397725.bronze.products")
df_cat = spark.table("evandb_7405616343397725.bronze.product_categories")
df_subcat = spark.table("evandb_7405616343397725.bronze.product_subcategories")
df_ret = spark.table("evandb_7405616343397725.bronze.returns")
df_sales = spark.table("evandb_7405616343397725.bronze.sales")
df_ter = spark.table("evandb_7405616343397725.bronze.territories")
```